



PRACTICA 1

Sistemas Distribuidos



19 DE MARZO DE 2024
EDUARD VERICAT | ALFONSO SANCHEZ

Índice

Objetivos de la práctica.	2
Ajuste de la fórmula de speedup a las métricas.	2
Limitaciones de CPU	2
XMLRPC	4
Un único nodo	4
Dos nodos.....	6
Tres nodos.....	8
Implementación y comportamiento con distintos nodos	10
PYRO4.....	11
Un único nodo	11
Dos nodos.....	12
Tres nodos.....	14
Grafica de speedup.....	15
RabbitMQ	16
SingleNode	16
Dos nodos.....	18
Tres nodos.....	20
REDIS.....	23
Un único nodo	23
Dos nodos.....	26
Tres nodos.....	28
Análisis de los 4 middleware	31
Análisis (1 nodo)	31
Análisis y conclusión de los speedup obtenidos	32
Escalado horizontal dinámico	32

Objetivos de la práctica.

Esta práctica está enfocada en analizar el rendimiento de diferentes middlewares mediante la creación de 2 servicios:

InsultService: que simplemente recibe insultos de forma remota y los guarda en una lista solo si no están allí. El servicio además ofrece mecanismos para obtener la lista de insultos, y un broadcaster que manda eventos aleatorios a los suscriptores cada 5 segundos.

InsultFilter: Es un servicio basado en la cola de trabajo. Este servicio permite a los clientes mandar textos que pueden tener o no insultos. El filtro reemplazará el insulto con “CENSURADO” y guardar el texto filtrado en una lista. El servicio además puede devolver el resultado si es necesario.

Ajuste de la fórmula de speedup a las métricas.

Para el cálculo del speedup la formula general es la siguiente.

$$S = \frac{T_1}{T_n}$$

Sin embargo, hemos adaptado la formula a las métricas usando la siguiente formula:

$$S = \frac{\text{Peticiones por segundo obtenidas}}{\text{Peticiones por segundo single node}}$$

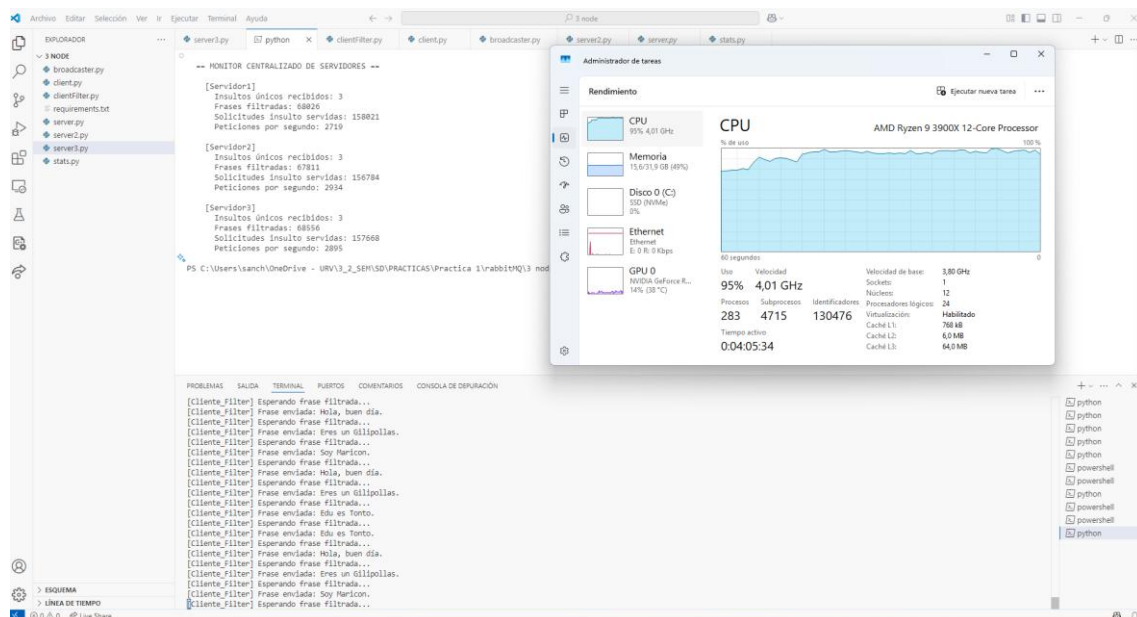
Esto nos permite poder estimar el speedup correctamente, ya que finalmente se obtiene la relación de aumento de rendimiento respecto a un único nodo.

Limitaciones de CPU

Debido a que la CPU es el cuello de botella, no se ha podido obtener mejores estadísticas, seguramente las capacidades sean superiores a las pruebas mostradas.

Pyro4 y RabbitMQ se han probado con un Ryzen 3900x.

XMLRPC y REDIS se han probado con un Ryzen 7800x.



De hecho, con un procesador de 12 cores 24 hilos ha sido llevado al máximo de su capacidad para algunas pruebas. Por lo que están totalmente condicionadas al procesador.

XMLRPC

Un único nodo

El sistema funciona con **un único servidor** que ofrece insultos a través de **XMLRPC**, un protocolo que permite hacer llamadas remotas como si fueran funciones locales.

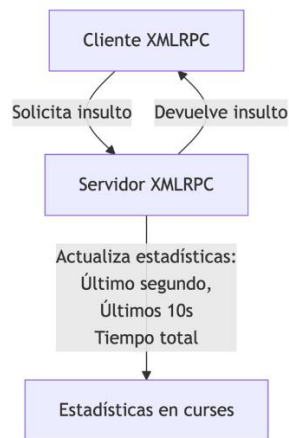
El **servidor** expone una función llamada `insultame()` que devuelve insultos desde una lista predefinida. Cada vez que alguien llama, le da uno y lo rota para que el siguiente reciba otro distinto. Además, lleva un contador de peticiones y muestra estadísticas en tiempo real con un panel hecho con `curses`.

Por otro lado, el **cliente** se conecta al servidor usando `http://localhost:8000` y empieza a pedir insultos en bucle. Cada vez que recibe uno, lo imprime por pantalla.

Todo es muy simple: una conexión directa, una función y un canal de ida y vuelta.
No hay intermediarios, colas ni nodos adicionales.

Total, de peticiones en un solo nodo:

- 5310 peticiones/segundo
- 54485 peticiones/10 segundos



En este esquema podemos ver el funcionamiento del **XML-RPC** con un único nodo. Primero, el cliente se conecta al servidor para realizar solicitudes, obtener insultos. El servidor procesa estas solicitudes y devuelve las respuestas correspondientes al cliente.

Además, el servidor actualiza estadísticas en tiempo real, como el número de solicitudes procesadas en el **último segundo**, en los **últimos 10 segundos** y el **total** desde el inicio. Estas estadísticas se muestran en una interfaz de consola utilizando **curses**, lo que permite monitorear el rendimiento del sistema de manera visual y dinámica.

También hemos realizado algunos tests.

```
✓ xmlrpc_node1
> ✓ server_test.py
```

Dos nodos

Ahora hemos simulado una arquitectura con **dos servidores XMLRPC** trabajando en paralelo. Cada nodo funciona de manera independiente: tiene su propia lista de insultos.

La idea es que los **clientes no siempre se conecten al mismo sitio**, sino que elijan aleatoriamente entre el **nodo 1 o el nodo 2** para cada petición. Así, conseguimos repartir mejor la carga y evitar que un solo servidor se sobrecargue cuando hay muchos clientes funcionando al mismo tiempo.

Gracias al **monitor**, podemos ver cuántas peticiones está manejando cada servidor en tiempo real (por segundo, por 10 segundos y total), además de una vista general combinada.

Al probarlo con varios clientes, se nota cómo el sistema va mucho más fluido que con un solo nodo. Las peticiones se reparten, las respuestas llegan más rápido, y se nota que el sistema es más estable bajo carga.

Estadísticas del Nodo 1:
Solicitudes en el último segundo: 1169
Solicitudes en los últimos 10 segundos: 22274

Estadísticas del Nodo 2:
Solicitudes en el último segundo: 1566
Solicitudes en los últimos 10 segundos: 13709

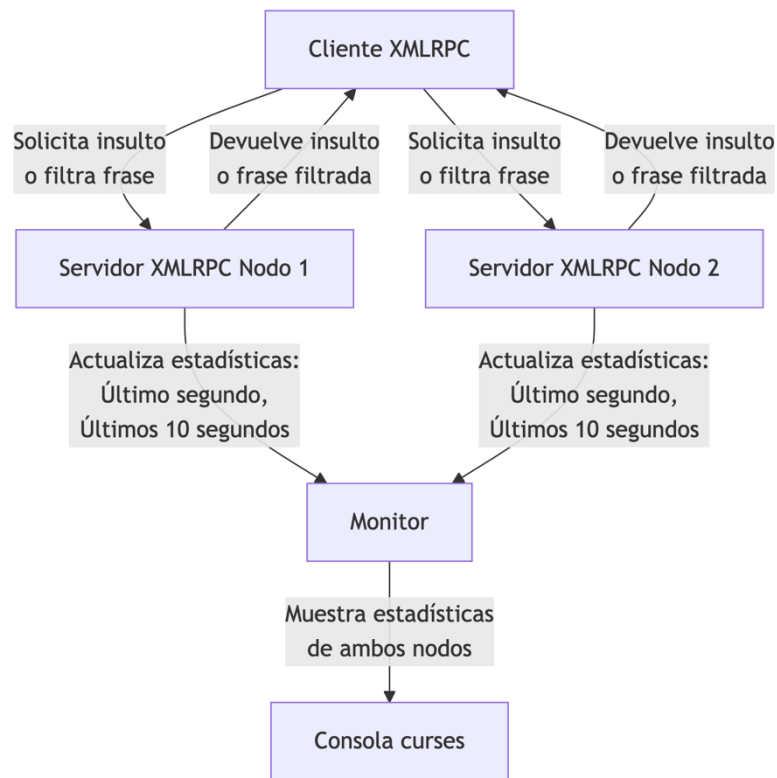
Estadísticas globales (suma de ambos nodos):
Solicitudes en el último segundo: 2735
Solicitudes en los últimos 10 segundos: 35983
Total de solicitudes desde el inicio: 147228

Terminal logs:

```
(task1_edu) evericat@edu: ~$ python3 xmllrpc_client.py
Conectando al servidor: http://localhost:8001/
Insulto recibido: despreciable
Frase filtrada: Eres un ****
Conectando al servidor: http://localhost:8002/
Insulto recibido: ignorante
Frase filtrada: Eres un ****
Conectando al servidor: http://localhost:8001/
Insulto recibido: cap de cul
Frase filtrada: Eres un ****
Conectando al servidor: http://localhost:8002/
Insulto recibido: menecato
Frase filtrada: Eres un ****
```

Total, de peticiones en un solo nodo:

- 2735 peticiones/segundo
- 35983 peticiones/10 segundos



El cliente envía solicitudes para obtener insultos o filtrar frases. Alterna entre el Nodo 1 y el Nodo 2 para repartir las peticiones de forma equilibrada. Recibe las respuestas de los nodos, ya sea un insulto o una frase filtrada.

Servidores (Nodo 1 y Nodo 2): Cada nodo trabaja de forma independiente, procesando las solicitudes que le llegan. Ambos nodos llevan un registro de las estadísticas: cuántas solicitudes han recibido en el último segundo, en los últimos 10 segundos y en total.

El **monitor** se conecta a los dos nodos para recopilar las estadísticas.

Muestra en tiempo real cuántas solicitudes está manejando cada nodo y también una vista general combinada. Esto permite ver claramente cómo se distribuyen las peticiones y cómo está funcionando el sistema.

Algunos de los tests realizados:

```
✓ ✔ xmlrpc_node2
> ✔ server1_test.py
> ✔ server2_test.py
```


Tres nodos

Ahora hemos ido un paso más allá, simulando un sistema con **tres servidores XMLRPC** funcionando en paralelo. Cada nodo sigue siendo independiente y mantiene su propia lista de insultos, gestionando sus propias peticiones sin depender de los otros dos.

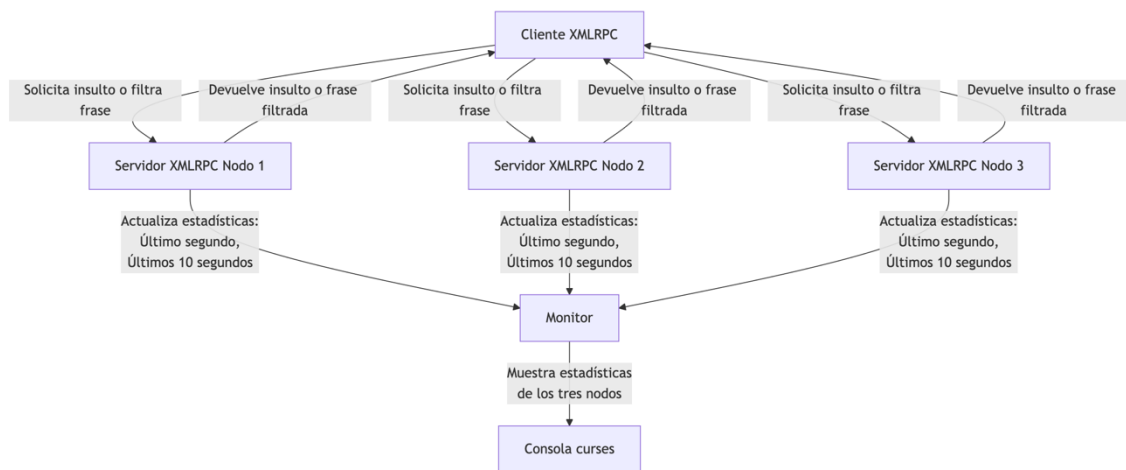
Los **clientes** siguen eligiendo aleatoriamente a qué nodo enviar cada solicitud, lo que permite repartir aún mejor la carga. Al tener un tercer nodo disponible, el sistema responde incluso mejor cuando hay muchos clientes conectados al mismo tiempo.

El **monitor** también ha sido actualizado para mostrar estadísticas de los tres nodos por separado y un resumen general, lo que facilita ver cómo se reparte el trabajo y si algún nodo está más cargado que los demás.

Durante las pruebas, con varios clientes lanzando peticiones a la vez, se ha notado una mejora clara respecto a la versión de 2 nodos. El sistema responde más rápido, se mantiene estable y puede absorber más carga sin saturarse.

Total, de peticiones en un solo nodo:

- 3727 peticiones/segundo
- 22533 peticiones/10 segundos

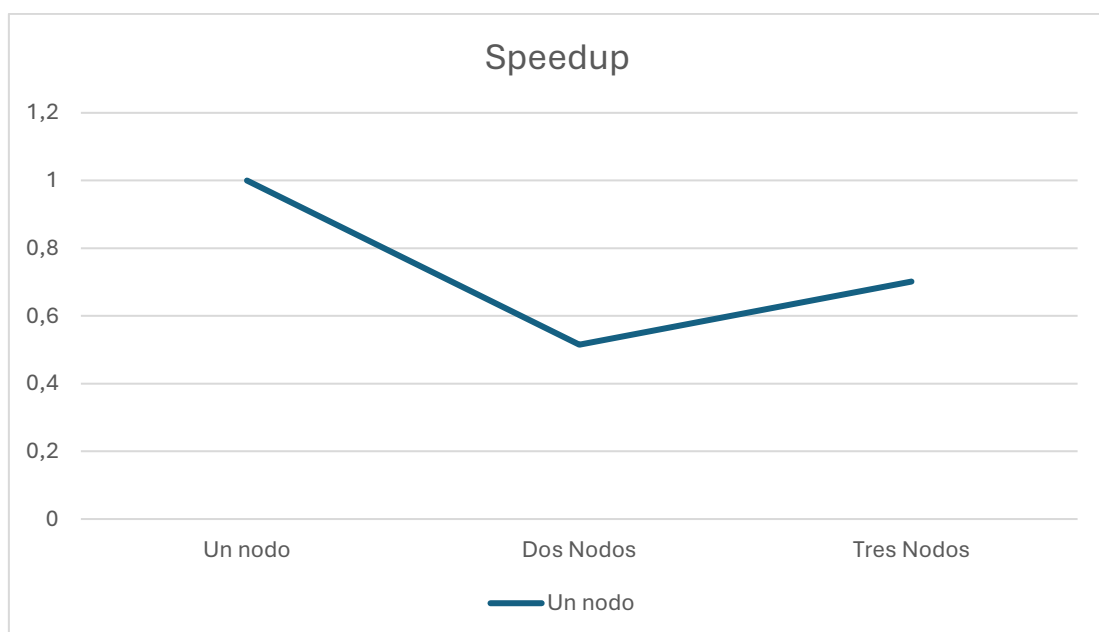


El sistema ahora cuenta con tres nodos XML-RPC trabajando en paralelo. Cada nodo tiene su propia lista de insultos y funciona de manera independiente, procesando las solicitudes que recibe del cliente. El cliente alterna entre los tres nodos para repartir las peticiones de forma equilibrada. Esto asegura que la carga se distribuya entre los nodos, evitando que uno solo se sobrecargue.

Gracias al monitor, podemos visualizar en tiempo real las estadísticas de cada nodo, como las solicitudes en el último segundo, en los últimos 10 segundos y el total acumulado. También hemos realizado algunos tests:

```

✓ xmlrpc_node3
> ✓ server1_test.py
> ✓ server2_test.py
> ✓ server3_test.py
  
```



Implementación y comportamiento con distintos nodos

Ahora vamos a tomar algunas conclusiones sobre el rendimiento en XMLRPC con múltiples nodos:

Aunque en muchos sistemas distribuir la carga entre varios nodos suele mejorar el rendimiento, en el caso de XMLRPC **esto no siempre se cumple**. De hecho, en nuestras pruebas hemos visto que al pasar de 1 nodo a 2 y luego a 3, el rendimiento general **empeora**, y esto tiene varias explicaciones técnicas y de diseño:

1. XMLRPC no está pensado para escalar horizontalmente

El protocolo XMLRPC está diseñado para establecer comunicación entre un cliente y un servidor de forma sencilla, pero **no incluye mecanismos nativos para distribuir trabajo entre varios servidores**, sincronizar estados o balancear la carga. Al añadir más nodos, no ganamos capacidad automáticamente, sino que añadimos complejidad.

2. Los clientes eligen el nodo al azar

En nuestra implementación, cada cliente elige aleatoriamente entre los nodos disponibles. Este sistema de distribución aleatoria **no garantiza un reparto equilibrado** de las peticiones. Es posible que dos clientes seguidos elijan el mismo nodo, mientras otros quedan inactivos, generando **cargas desiguales y cuellos de botella innecesarios**.

Además, al no haber un control de disponibilidad o de carga, los clientes **pueden quedar esperando** a un nodo que no responde a tiempo.

3. Los nodos son totalmente independientes

Los nodos **no colaboran entre sí** ni reparten tareas de forma eficiente. Todos hacen exactamente lo mismo, pero sin coordinación, lo que lleva a **duplicar esfuerzos sin mejorar resultados**.

4. Añadir más nodos implica más puntos de espera

Con más nodos activos, también hay más claves y canales que monitorizar (más `request_insult_nodeX`, más `response_filter_nodeX`, etc.). Esto introduce más lógica en los clientes, **más tiempo de espera entre envíos y respuestas**.

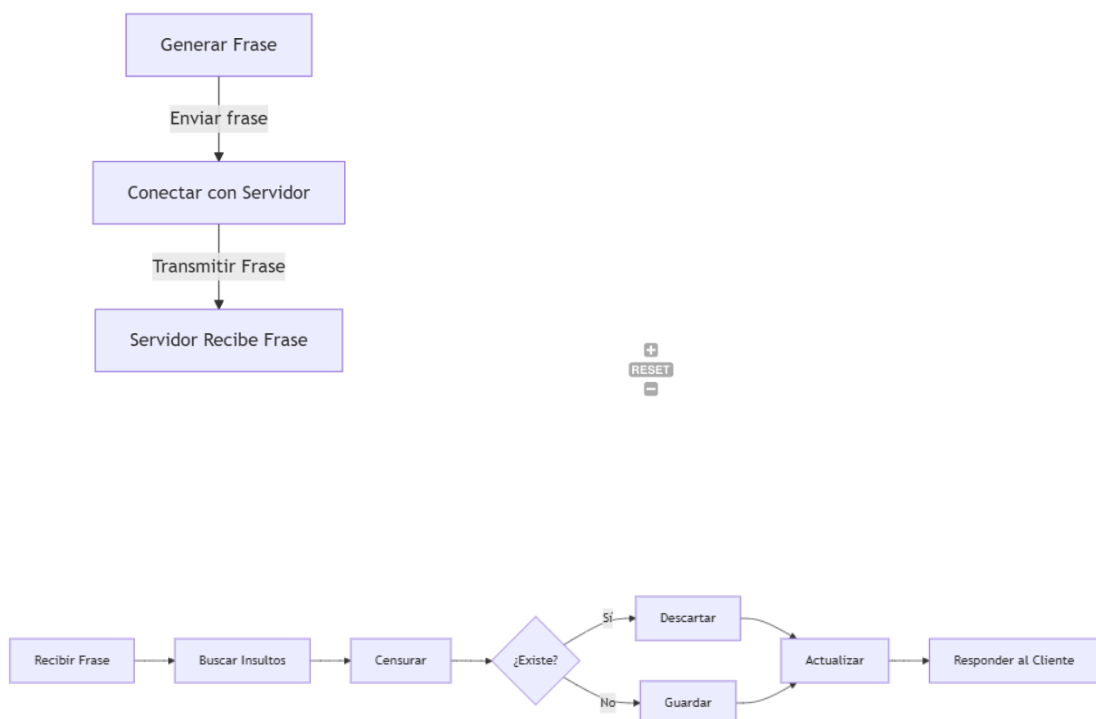
En conclusión, en entornos simples y con pocos clientes, **un único nodo XMLRPC funciona mejor** porque es directo, rápido y evita la sobrecarga de decisiones y espera. Al pasar a 2 o 3 nodos, **el sistema se complica innecesariamente**, sin un verdadero balanceo de carga ni colaboración entre servidores.

PYRO4

Un único nodo

Como se comentó en clase, en XMLRPC/Pyro4 no es necesario hacerlo tan complejo como REDIS o RabbitMQ. Por ello hemos implementado el siguiente esquema, que tiene complejidad para poder saturar el servidor y revisar cuanto es capaz de soportar para calcular su speedup.

Implementación del insult filter:



Se añade un procesamiento extra en el servidor donde ahora le llega una frase con un posible insulto, debe analizar la frase y censurar el insulto antes de guardarla. Además, antes de guardar la frase se debe verificar si esta ya existe en la lista del servidor.

Obtenemos el siguiente rendimiento con 9 clientes funcionando de forma simultánea para generar el máximo estrés al servidor.

```
PROBLEMAS  SALIDA  TERMINAL  PUERTOS  COMENTARIOS  CONSOLA DE DEPURACIÓN
Insultos enviados/segundo: 3195
Insultos enviados/10 segundos: 29949
Insultos enviados/1 minuto: 198331
Presiona 'q' para salir.
```

Rendimiento: 3195 peticiones / segundo.

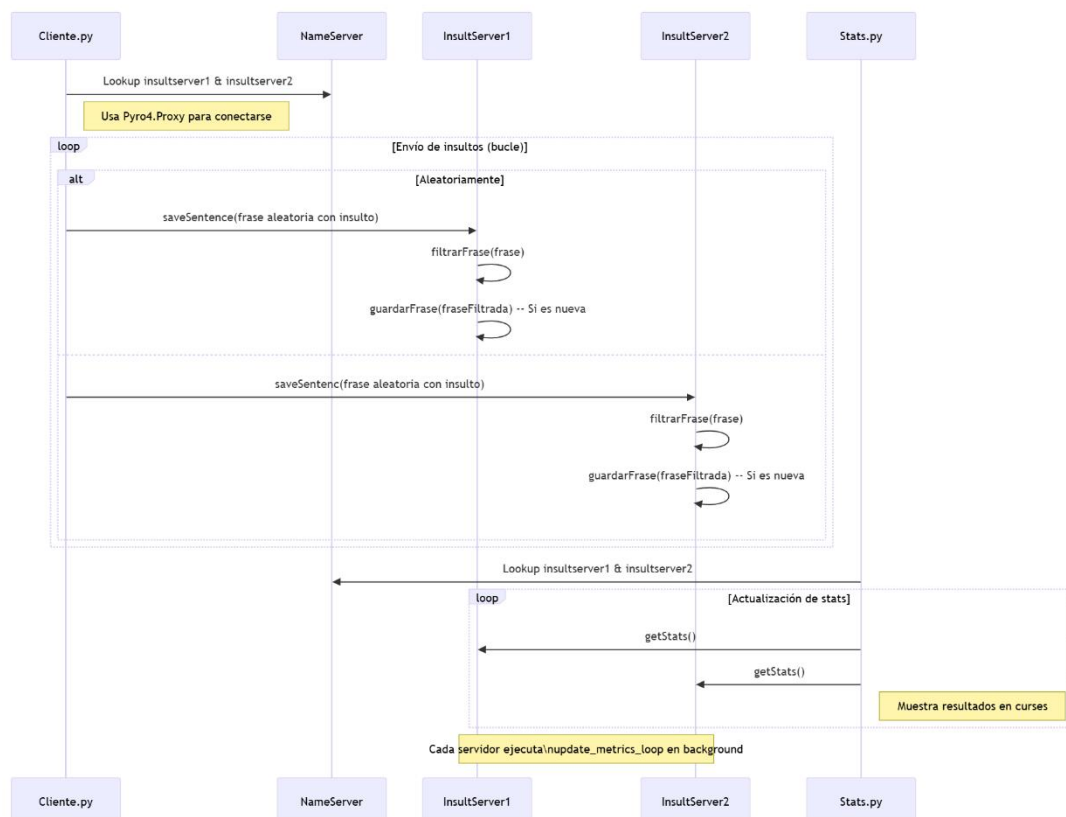
Se ha realizado las pruebas unitarias solicitadas para verificar el correcto funcionamiento de el código del middleware Pyro4 con 1 solo nodo.

Dos nodos

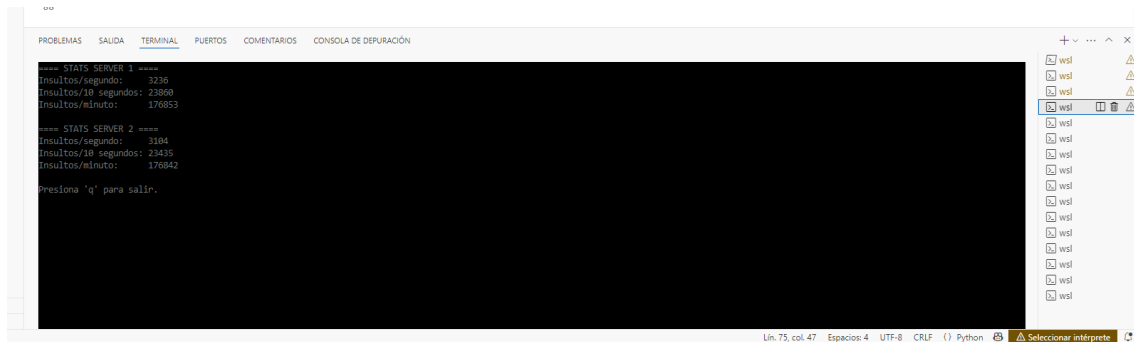
Para elaborar el código con 2 nodos, se ha pensado en un escalado horizontal estático. Donde los clientes mandarán peticiones de manera aleatoria a uno de los 2 servidores disponibles en su listado de servidores.

De esta forma los 2 servidores se reparten la carga por igual para los clientes. El resultado finalmente es prácticamente una triplicación del numero de peticiones que soporta.

A continuación, se adjunta un diagrama de secuencia (general) del código.



Finalmente, ejecutando 12 clientes para forzar la carga lo máximo posible, obtenemos las siguientes estadísticas.



```
==== STATS SERVER 1 ====
Resultos/segundo: 3236
Resultos/10 segundos: 23360
Resultos/minuto: 17683

==== STATS SERVER 2 ====
Resultos/segundo: 3184
Resultos/10 segundos: 23435
Resultos/minuto: 176842

Presiona 'q' para salir.
```

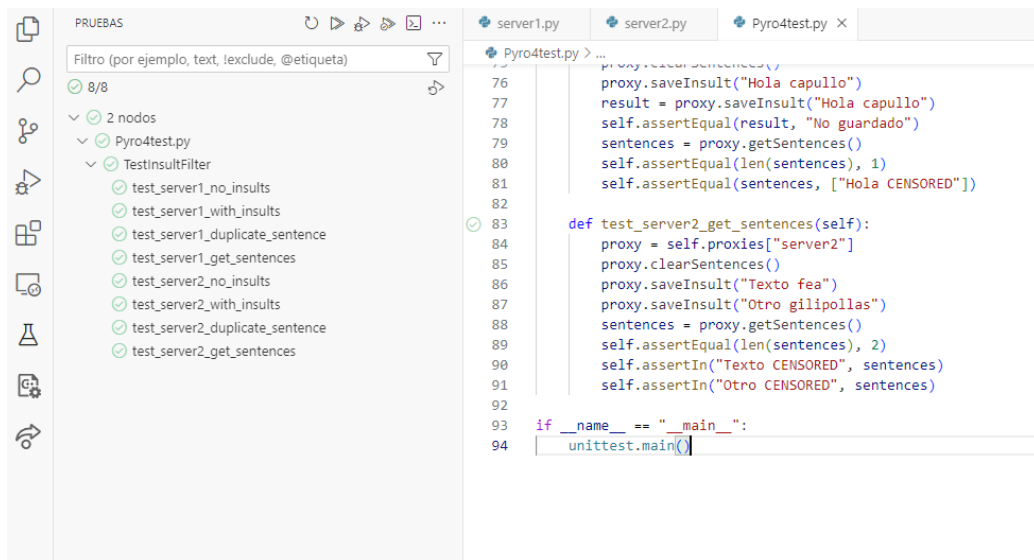
Total, peticiones por segundo: 6340 peticiones/segundo

Finalmente, el tener 2 nodos nos arroja el siguiente speedup.

$$S = \frac{6340}{3195} \approx 1.98$$

Es decir, estamos obteniendo un rendimiento 0.98 veces superior con 2 nodos respecto a las pruebas con 1 único nodo. Es decir, un 98% o el doble prácticamente de rendimiento.

También por supuesto, hemos realizado las pruebas unitarias mediante el uso de unittest para poder verificar el funcionamiento de los 2 nodos.

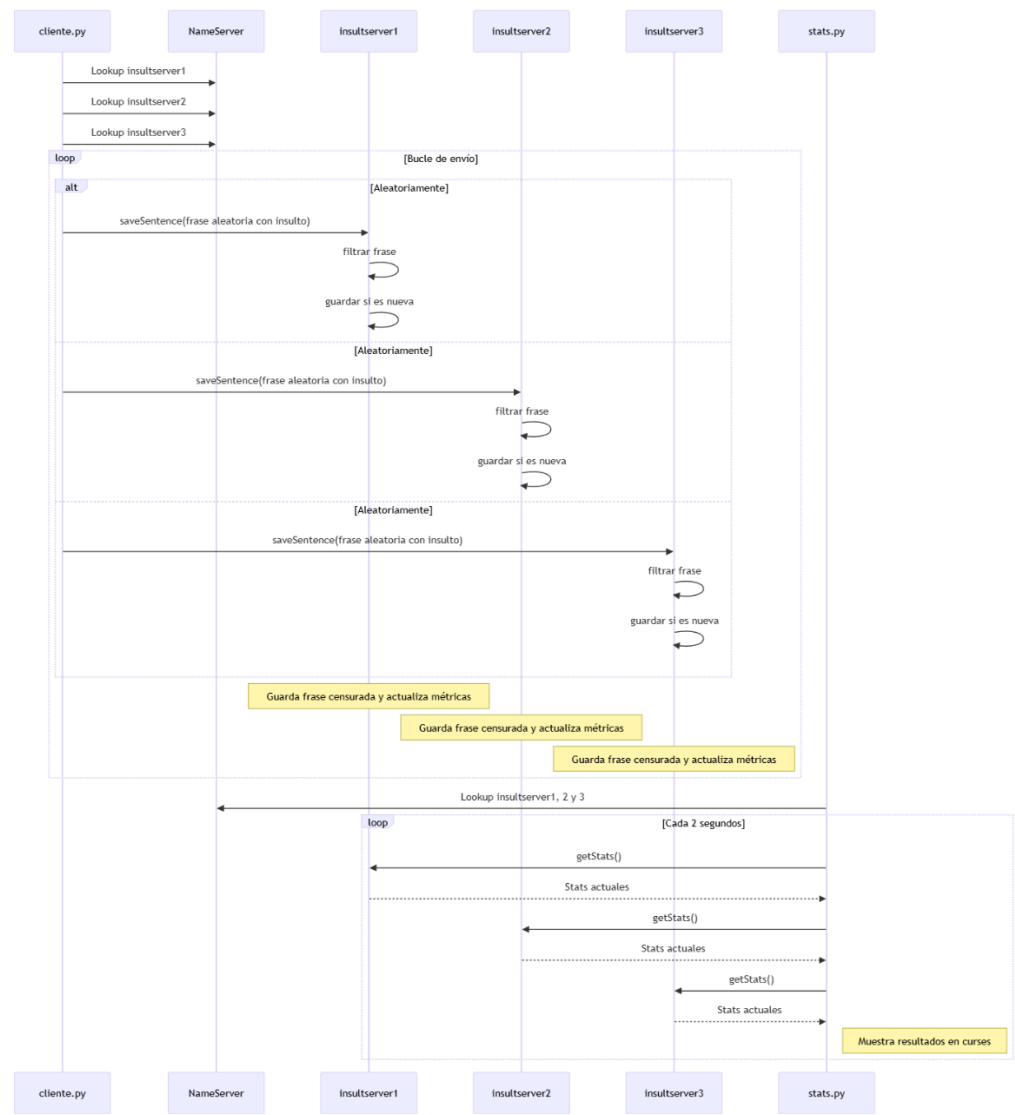


```
PRUEBAS
Filtro (por ejemplo, text, lexclude, @etiqueta)
8/8
2 nodos
Pyro4test.py
TestInsultFilter
test_server1_no_insults
test_server1_with_insults
test_server1_duplicate_sentence
test_server1_get_sentences
test_server2_no_insults
test_server2_with_insults
test_server2_duplicate_sentence
test_server2_get_sentences

Pyro4test.py
76 proxy.saveInsult("Hola capullo")
77 result = proxy.saveInsult("Hola capullo")
78 self.assertEqual(result, "No guardado")
79 sentences = proxy.getSentences()
80 self.assertEqual(len(sentences), 1)
81 self.assertEqual(sentences, ["Hola CENSORED"])
82
83 def test_server2_get_sentences(self):
84 proxy = self.proxies["server2"]
85 proxy.clearSentences()
86 proxy.saveInsult("Texto fea")
87 proxy.saveInsult("Otro gilipollas")
88 sentences = proxy.getSentences()
89 self.assertEqual(len(sentences), 2)
90 self.assertIn("Texto CENSORED", sentences)
91 self.assertIn("Otro CENSORED", sentences)
92
93 if __name__ == "__main__":
94 unittest.main()
```

Tres nodos.

El escalado a 3 nodos es idéntico con 2 nodos. Todo el sistema funciona exactamente de la misma manera. Igualmente, para poder entender los cambios, se adjunta el siguiente diagrama de secuencia.



Finalmente se han obtenido las siguientes estadísticas tras probar la ejecución con 18 clientes activos.

```
PROBLEMAS  SALIDA  TERMINAL  PUERTOS  COMENTARIOS  CONSOLA DE DEPURACIÓN

==== STATS SERVER 1 ====
Insultos/segundo: 2970
Insultos/10 segundos: 22884
Insultos/minuto: 127119

==== STATS SERVER 2 ====
Insultos/segundo: 3023
Insultos/10 segundos: 22943
Insultos/minuto: 127589

==== STATS SERVER 3 ====
Insultos/segundo: 3079
Insultos/10 segundos: 22998
Insultos/minuto: 127935

Total insultos/segundo: 9072

Presiona 'q' para salir.
```

Sumando un total de 9072 peticiones / segundo y dando el siguiente speedup.

$$S = \frac{9072}{3195} \approx 2.83$$

En este caso respecto a 2 nodos hemos obtenido un mayor rendimiento, dando la siguiente diferencia (aumento):

$$\text{Rendimiento ganado respecto 2 nodos} = 2.83 - 1.98 = 0,85 * 100 = 85\%$$

$$\text{Rendimiento ganado respecto 1 nodo} = 2.83 - 1 = 1.83 * 100 = 183\%$$

También se ha realizado pruebas unitarias donde se verifica el correcto funcionamiento de los 3 nodos:

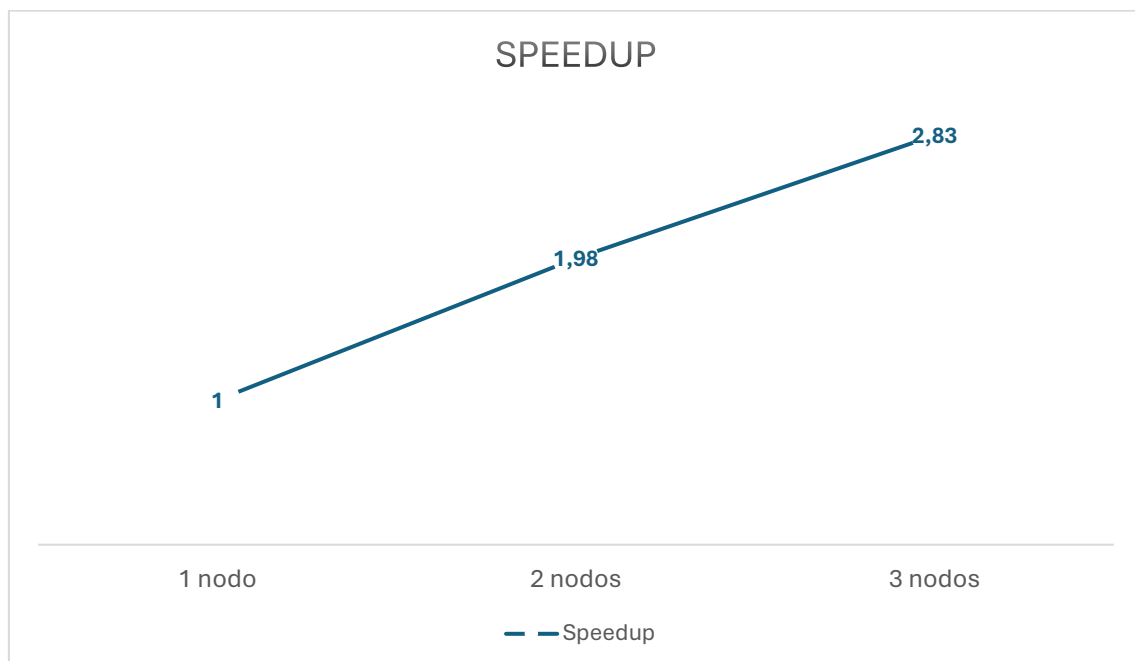
The screenshot shows a PyCharm IDE with a test runner window on the left and a code editor on the right. The test runner shows 12 tests passed in 15.46s. The code editor shows the implementation of the `InsultServer` class, which handles insults and metrics for three nodes.

```

12
13 self.insults_per_sec = 0
14 self.insults_per_10s = 0
15 self.insults_per_min = 0
16
17 self.lock = threading.Lock()
18
19 # Iniciar hilo de métricas
20 metrics_thread = threading.Thread(target=self.update_metrics_loop)
21 metrics_thread.daemon = True
22 metrics_thread.start()
23
24 def saveInsult(self, sentence):
25     result = "No guardado"
26     sentence = sentence.strip()
27
28     words = sentence.split()
29     censored_words = ["CENSORED" if word.lower() in self.insults else word for word in words]
30     filterSentence = " ".join(censored_words)
31
32
33 PROBLEMAS SALIDA TERMINAL RESULTADOS DE LA PRUEBA PUERTOS COMENTARIOS CONSOLA DE DEPURACIÓN
Received test ids from temp file.
test_server1_duplicate_sentence (Pyro4test.TestInsultFilter.test_server1_duplicate_sentence) ... ok
test_server1_get_sentences (Pyro4test.TestInsultFilter.test_server1_get_sentences) ... ok
test_server1_no_insults (Pyro4test.TestInsultFilter.test_server1_no_insults) ... ok
test_server1_with_insults (Pyro4test.TestInsultFilter.test_server1_with_insults) ... ok
test_server2_duplicate_sentence (Pyro4test.TestInsultFilter.test_server2_duplicate_sentence) ... ok
test_server2_get_sentences (Pyro4test.TestInsultFilter.test_server2_get_sentences) ... ok
test_server2_no_insults (Pyro4test.TestInsultFilter.test_server2_no_insults) ... ok
test_server2_with_insults (Pyro4test.TestInsultFilter.test_server2_with_insults) ... ok
test_server3_duplicate_sentence (Pyro4test.TestInsultFilter.test_server3_duplicate_sentence) ... ok
test_server3_get_sentences (Pyro4test.TestInsultFilter.test_server3_get_sentences) ... ok
test_server3_no_insults (Pyro4test.TestInsultFilter.test_server3_no_insults) ... ok
test_server3_with_insults (Pyro4test.TestInsultFilter.test_server3_with_insults) ... ok
.....
Ran 12 tests in 15.050s
OK
Finished running tests!

```

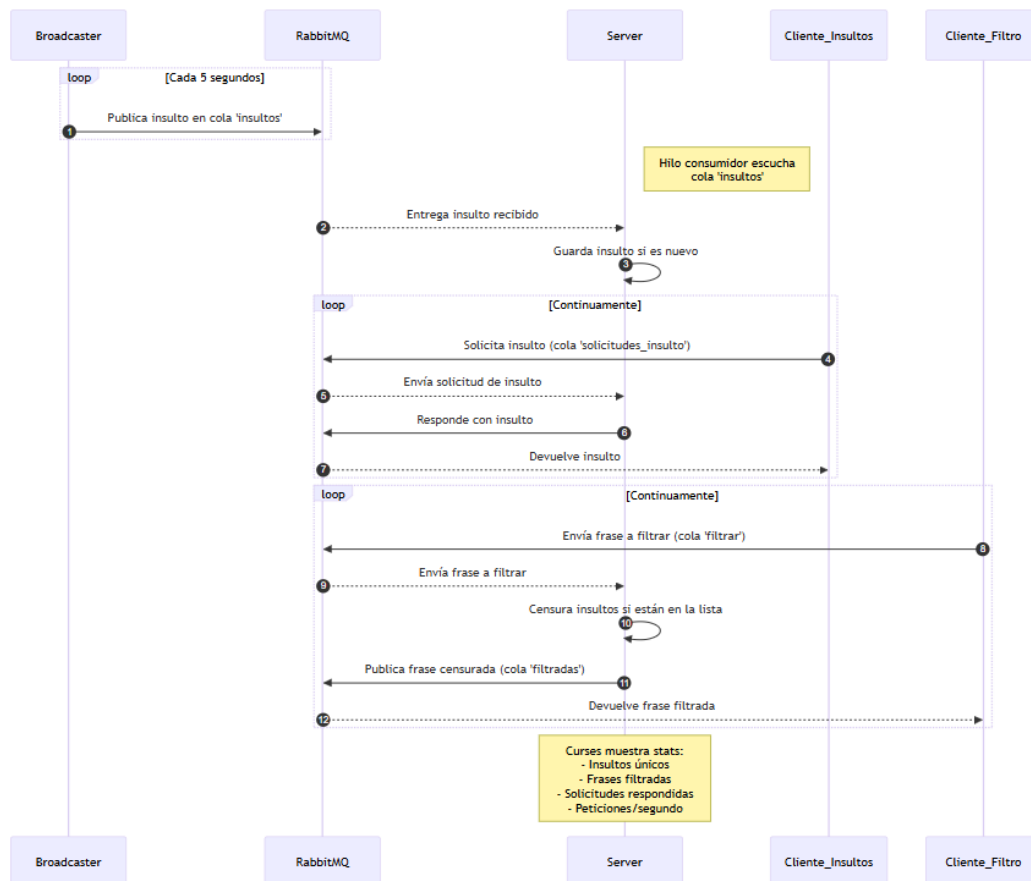
Grafica de speedup.



RabbitMQ

SingleNode

Esquema seguido de programación:



Como se observa en el esquema tenemos un broadcaster que cada 5 segundos mandara un nuevo insulto al servidor mediante una cola de insultos. Luego en el servidor tenemos 3 colas:

- Cola solicitudes_insultos donde se envían y reciben insultos por parte del cliente (hecho así para probar el rendimiento de una sola cola).
- Cola de filtrar, donde se manda una frase con un posible insulto.
- Cola de filtrada donde el servidor devuelve la frase filtrada.

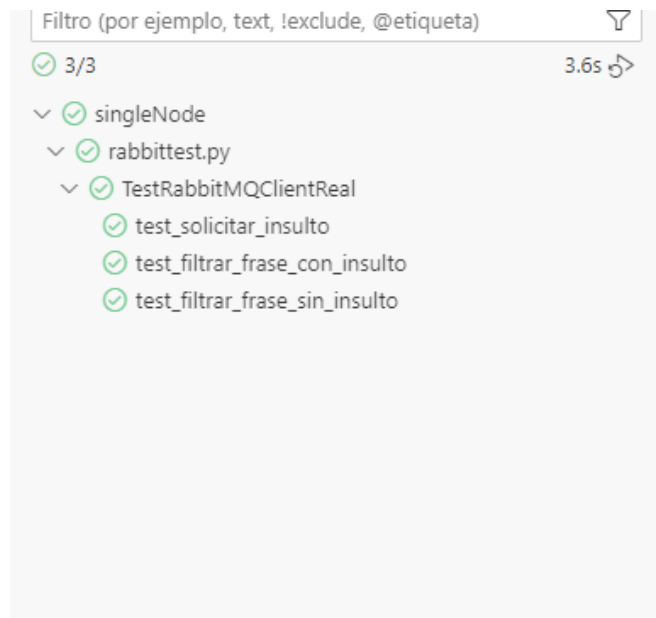
Hemos visto que, en el caso de usar una única cola, el rendimiento es inferior, pues conseguimos máximo 10-30 solicitudes/segundo y consideramos que realmente no es la forma que trabajar con rabbit. Solo que se ha hecho en este apartado concreto para probar y ver que ocurría a modo práctica (luego con 2 y 3 nodos esto se ha corregido).

Estadísticas obtenidas:

```
== Servidor RabbitMQ - Estadísticas ==  
  
Insultos únicos recibidos: 3  
Frases filtradas: 208613  
Solicitudes insulto servidas: 0  
Peticiones por segundo (total): 5287  
  
Ejemplo insultos: Maricon, Tonto., Gilipollas.
```

Teniendo 5287 peticiones por segundo, sería la base para aplicar la formula del speedup.

Finalmente, también hemos hecho pruebas unitarias básicas comprobando el funcionamiento de los métodos de insultar y filtrar del servidor mediante el uso de las colas de rabbitmq.

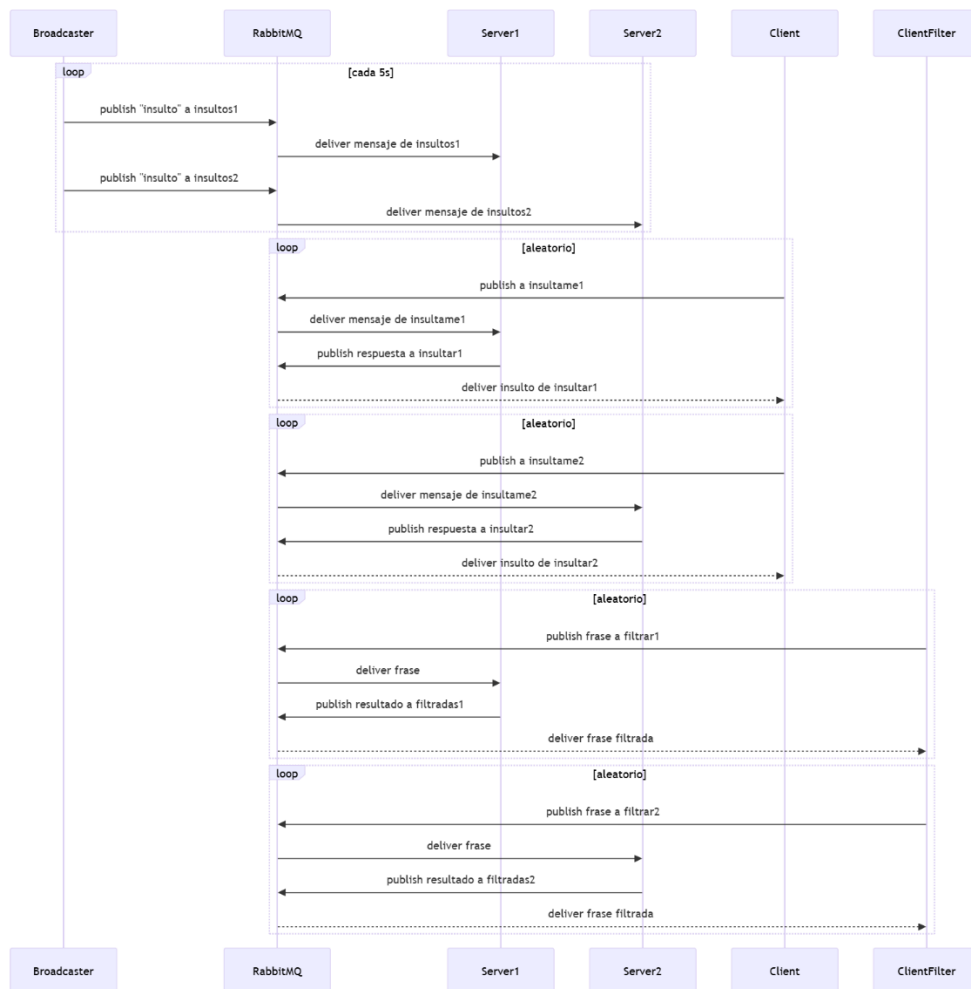


Dos nodos

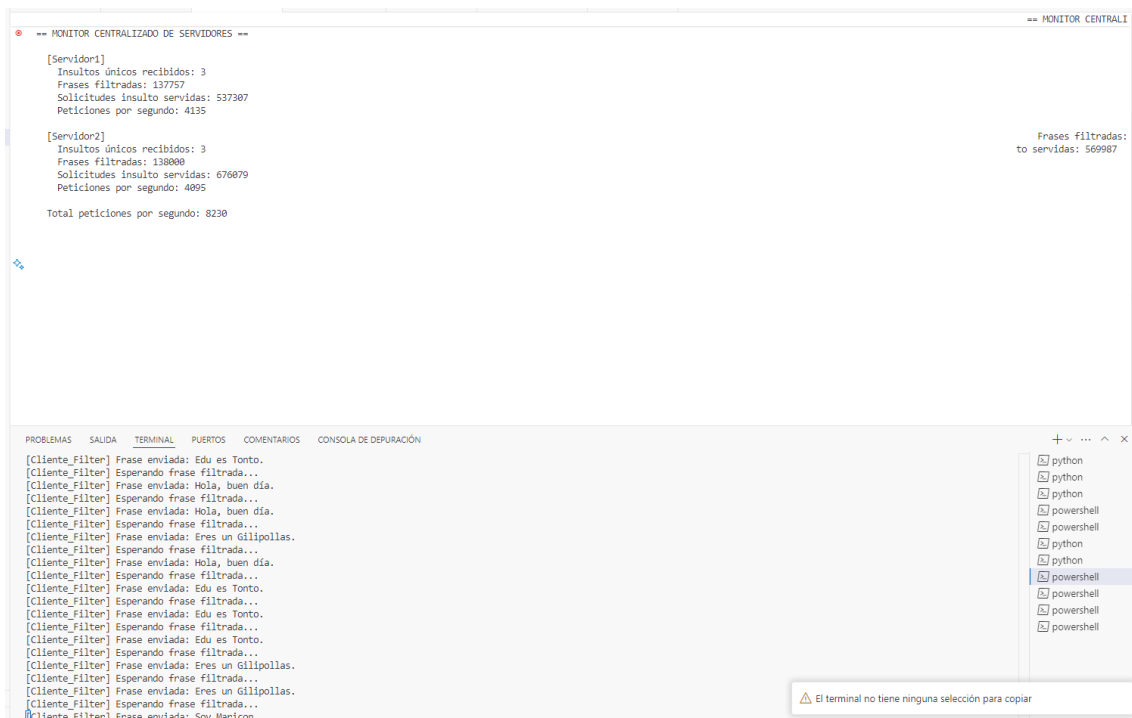
Para poder escalar a 2 nodos, deberemos tener ahora el doble de colas, en total tendremos 10 colas. Por servidor:

- 2 colas correspondientes para el filtrado de frases.
- 2 colas correspondientes para las peticiones de insulto
- 1 cola correspondiente para el broadcaster.

Esquema del código:



Estadísticas obtenidas:

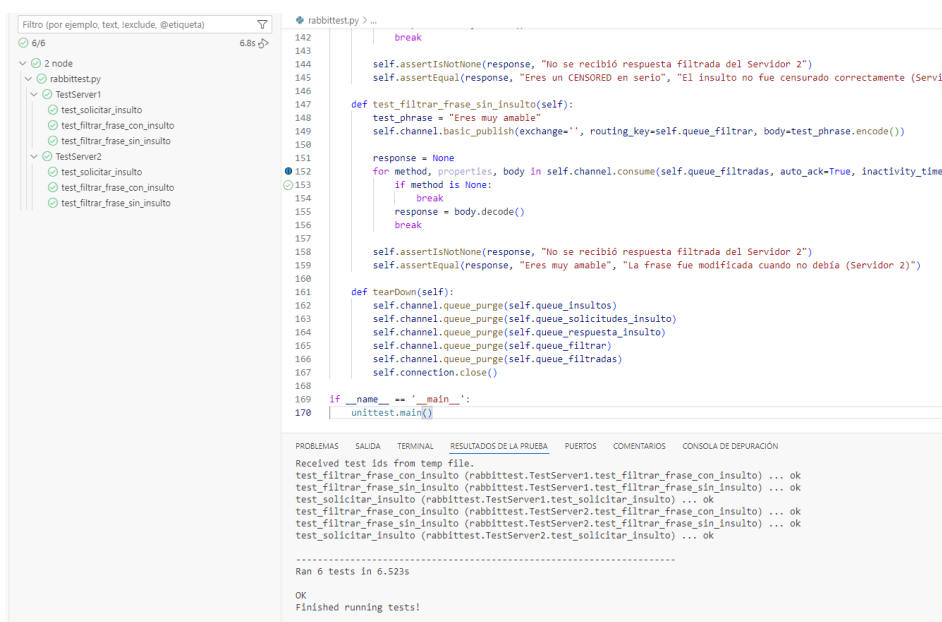


En total obtenemos 8230 peticiones / segundo con algunos picos incluso cercanos a 9000 peticiones / segundo. Pero tomaremos esta medida para los cálculos.

$$S = \frac{8230}{5287} \approx 1.55$$

Esto nos da una diferencia de rendimiento de $1.55 - 1 = 0.55 = 55\%$ más respecto a un nodo.

Hemos implementado pruebas básicas de funciones de ambos servidores, dando todas las pruebas correctas.

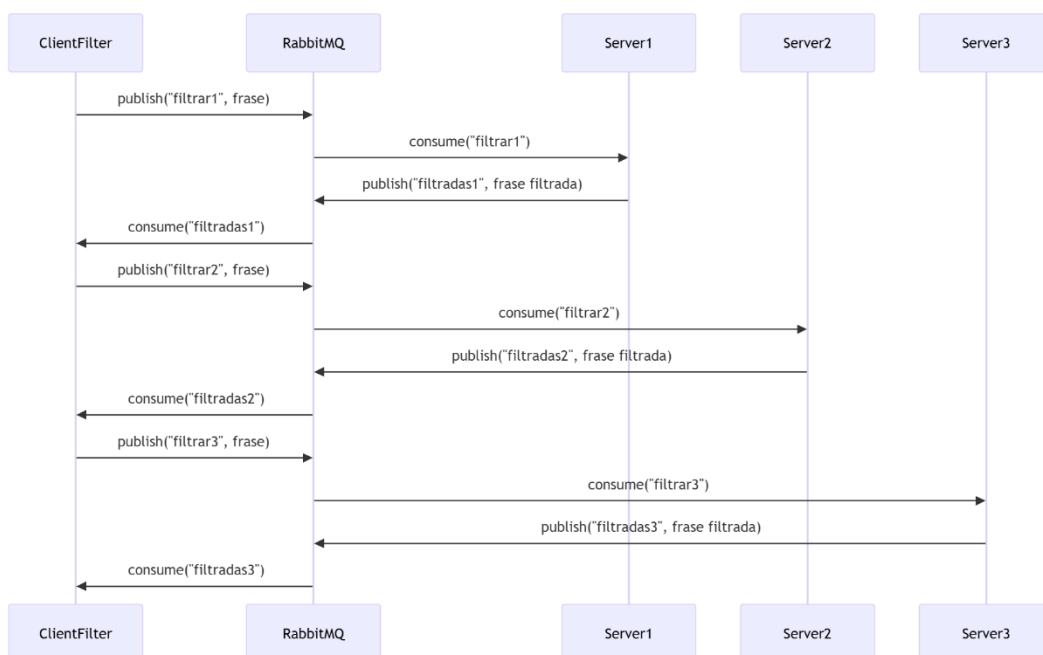
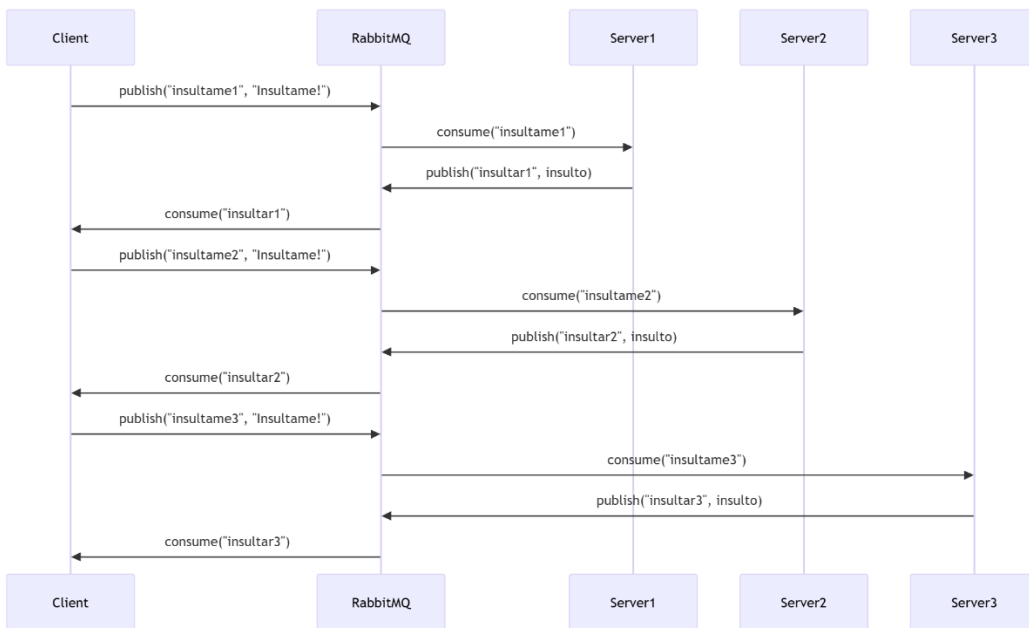
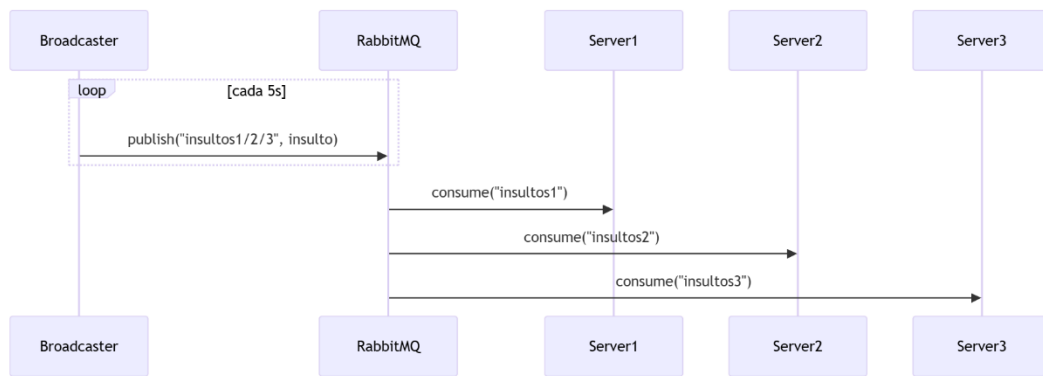


Tres nodos

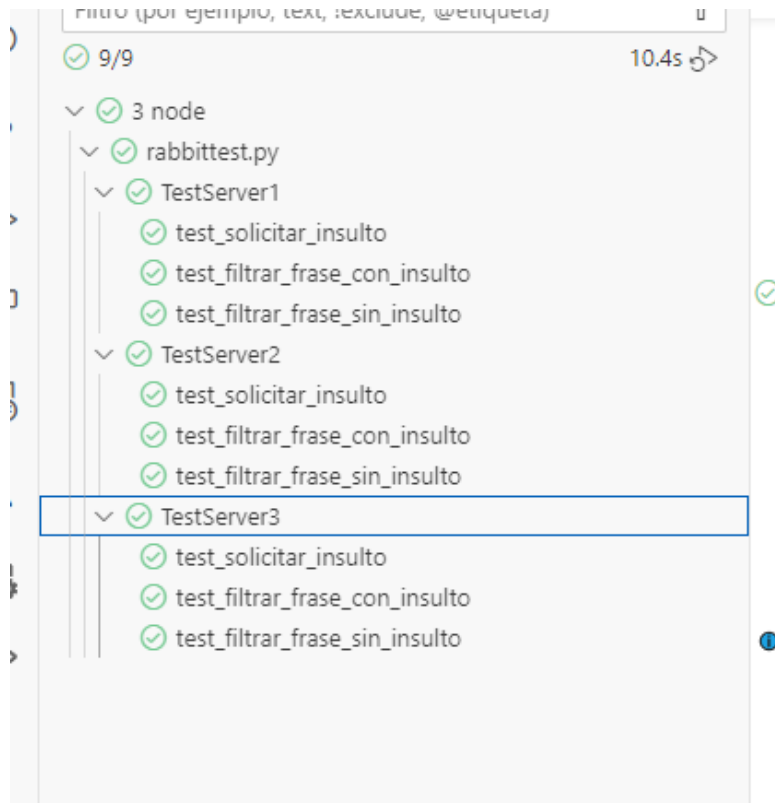
Finalmente tenemos el código con 3 nodos. Es exactamente que con 2 nodos simplemente ahora tenemos 3 servidores con el consecuente incremento en el número de colas.

Para equilibrar más las peticiones se ha procedido que los clientes ahora apliquen un round robin en vez de una selección aleatoria sobre la lista de servidores. De esta manera el reparto de peticiones es equitativa.

A continuación, presentamos varios diagramas de secuencia de interacción entre todos los componentes:



También por supuesto hemos realizado las pruebas, pero ahora para los 3 nodos.



Resultado de RabbitMQ con 3 nodos.

```
== MONITOR CENTRALIZADO DE SERVIDORES ==

[Servidor1]
Insultos únicos recibidos: 3
Frases filtradas: 111535
Solicitudes insulto servidas: 611093
Peticones por segundo: 4170

[Servidor2]
Insultos únicos recibidos: 3
Frases filtradas: 112306
Solicitudes insulto servidas: 628210
Peticones por segundo: 4368

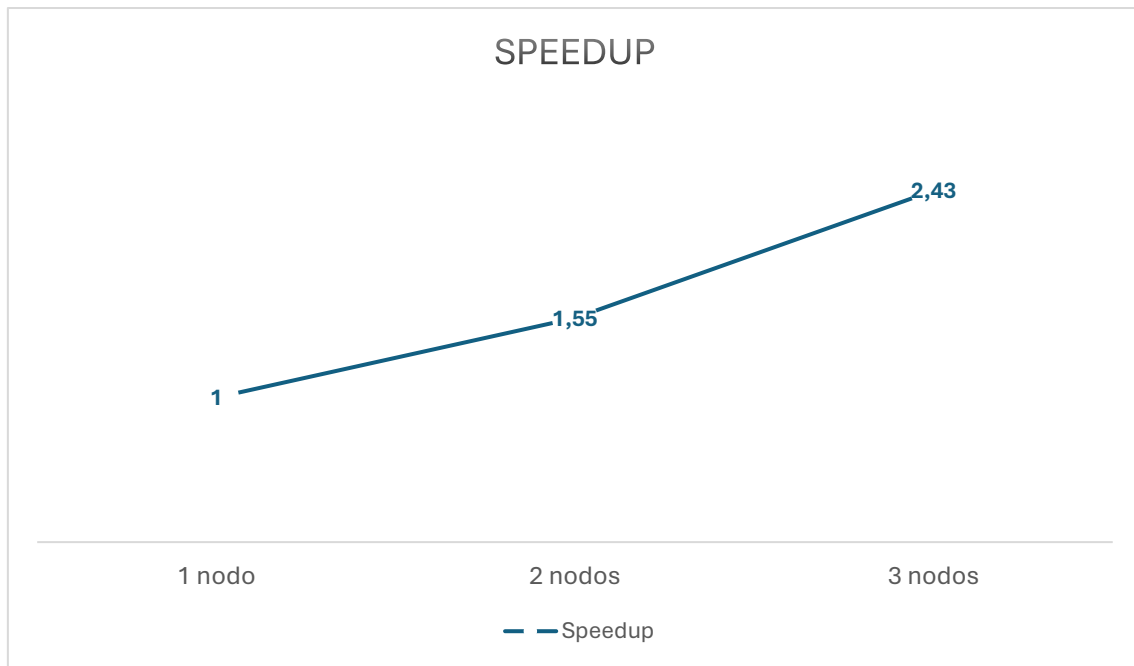
[Servidor3]
Insultos únicos recibidos: 3
Frases filtradas: 111440
Solicitudes insulto servidas: 616844
Peticones por segundo: 4323

Total peticiones por segundo (3 servidores): 12861
```

Total, medido: 12861 peticiones / segundo.

$$S = \frac{12861}{5287} \approx 2.43$$

Esto nos da un rendimiento mejorado respecto a un nodo de $2.43 - 1 = 1.43 = 143\%$.



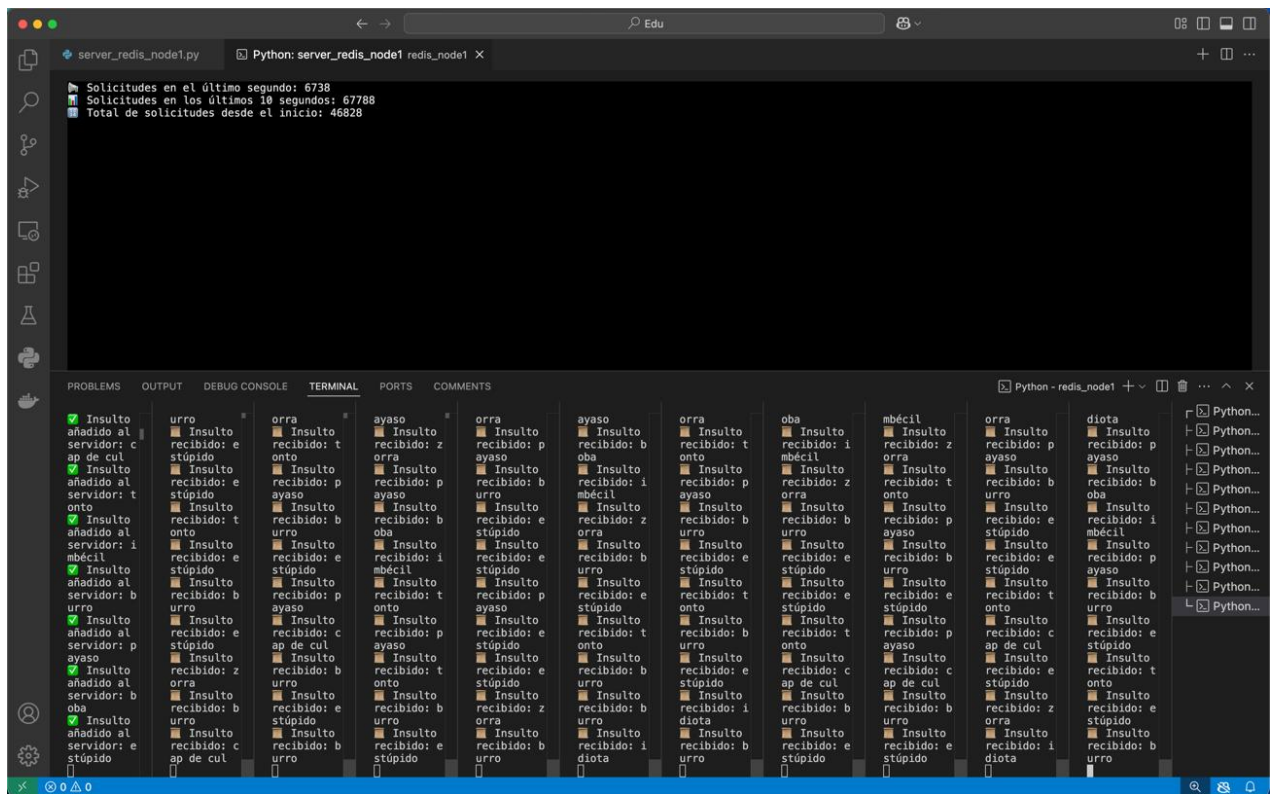
REDIS

Un único nodo

En esta parte del proyecto hemos creado un sistema distribuido usando **Redis como middleware**. Todo está montado sobre un único nodo Redis, que se encarga de coordinar la comunicación entre varios procesos: un *broadcaster*, un *servidor*, y dos tipos de *clientes*.

La idea principal es usar Redis no solo para almacenar datos, sino también para intercambiar mensajes entre procesos mediante claves temporales. Esto nos ha permitido simular un sistema con distintos servicios que se comunican de forma sencilla y rápida.

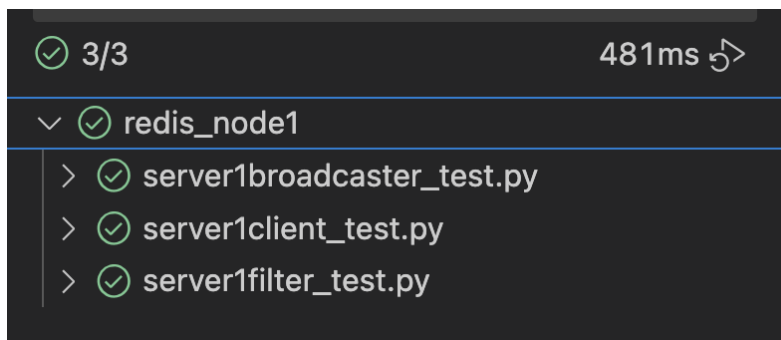
Obtenemos el siguiente rendimiento con 10 clientes funcionando simultáneamente para generar el máximo estrés al servidor:

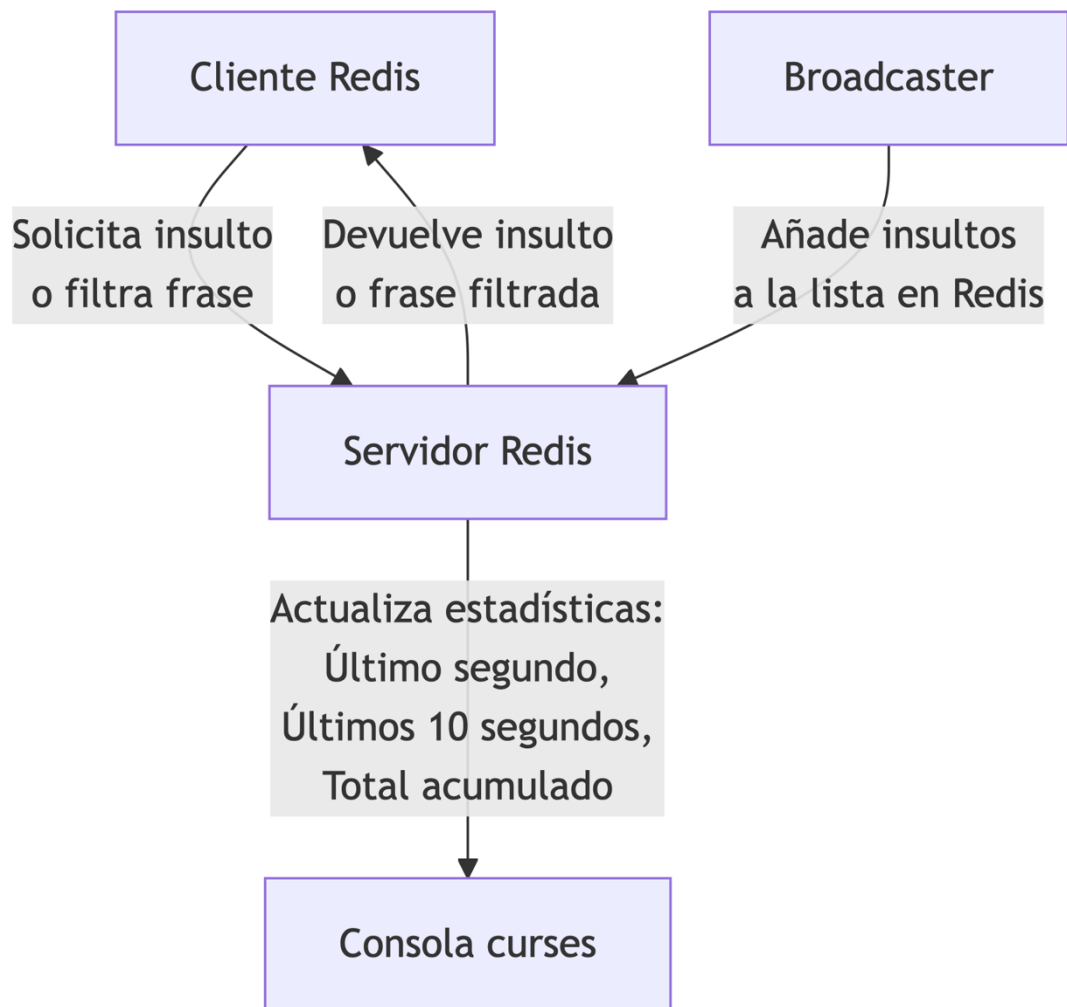


Total, de peticiones en un solo nodo:

- 6738 peticiones/segundo
- 67788 peticiones/10 segundos

Algunos de los tests realizados





El esquema muestra cómo funciona el sistema Redis con un solo nodo. Los clientes envían solicitudes al servidor para obtener insultos o filtrar frases, y el servidor responde procesando estas peticiones. El broadcaster se encarga de mantener la lista de insultos actualizada en Redis. Además, el servidor actualiza estadísticas en tiempo real, que se muestran en una consola interactiva utilizando curses.

Dos nodos

Para escalar el sistema, hemos diseñado una solución basada en **escalado horizontal estático**, en la que se ejecutan **dos servidores independientes** actuando como nodos. Cada uno se encarga de procesar peticiones por separado, permitiendo repartir la carga entre ambos.

Los clientes no están ligados a un nodo específico, sino que **envían cada petición a uno de los dos nodos de forma aleatoria**, simulando así un sistema distribuido con balanceo simple. Esto permite que los dos servidores trabajen en paralelo, mejorando considerablemente el rendimiento general.

Además, el **broadcaster** se ha modificado para enviar insultos a ambos nodos a la vez, asegurando que los dos tengan la misma base de datos para trabajar.

También se ha añadido un **monitor en tiempo real**, que muestra por pantalla las estadísticas de cada nodo (peticiones por segundo, por 10 segundos y total acumulado), así como un resumen general combinando ambos.

```
Estadísticas del Nodo 1:
Solicitudes en el último segundo: 6317
Solicitudes en los últimos 10 segundos: 3692
Total de solicitudes desde el inicio: 321789

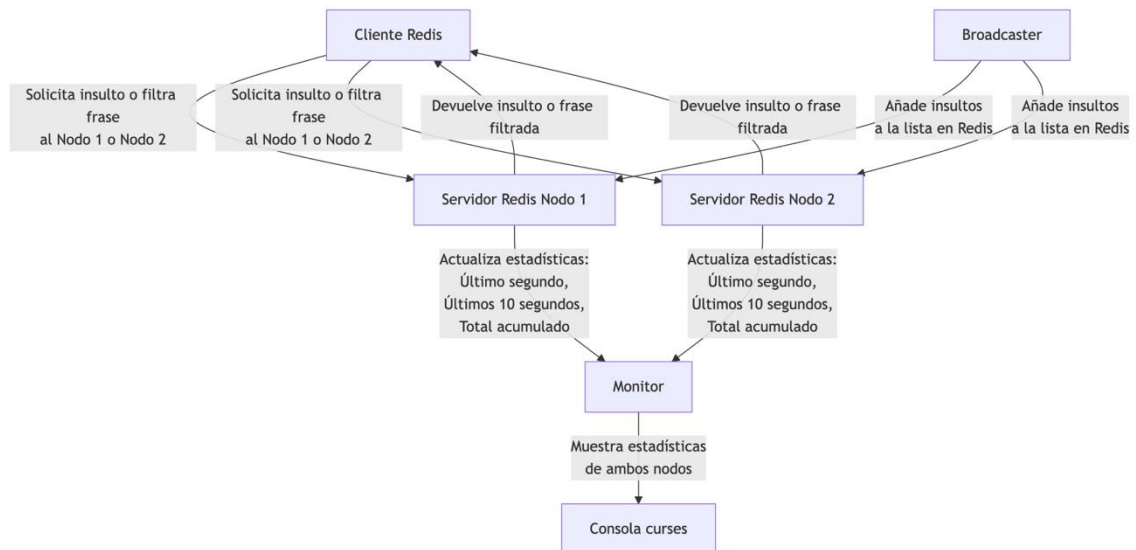
Estadísticas del Nodo 2:
Solicitudes en el último segundo: 6897
Solicitudes en los últimos 10 segundos: 5589
Total de solicitudes desde el inicio: 321704

Estadísticas generales (suma de ambos nodos):
Solicitudes en el último segundo: 1240
Solicitudes en los últimos 10 segundos: 92812
Total de solicitudes desde el inicio: 643493
```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS	COMMENTS
desalmado +	del node1:	o insulto	del node1:	recibido	o insulto
añadido a	desgracia	al node2..	al node2..	al node2..	al node2..
l Nodo 2:	do	Solicitand	imbecil	del node2:	Insulto
desalmado	o insulto	al node2..	Solicitand	al node2..	recibido
añadido a	al node2..	al node1..	o insulto	al node2..	del node1:
l Nodo 1:	.	Solicitand	o insulto	Solicitand	ridículo
egosta	Solicitand	al node2..	.	al node2..	o insulto
añadido a	o insulto	al node2..	Insulto	al node2..	del node2:
l Nodo 2:	al node2..	.	recibido	al node2..	mentecato
egosta	.	Solicitand	al node2:	al node2..	o insulto
añadido a	zsh: suspe	al node1..	traidor	al node2..	al node2..
l Nodo 1:	nded pyth	Solicitand	Solicitand	al node2..	al node2..
añadido a	on3 client	al node1..	al node2..	al node2..	al node2..
l Nodo 1:	_redis.py	o insulto	al node1..	al node2..	al node2..
farsante	(task1_edu	al node1..	.	al node2..	al node2..
Insulto	(task1_edu	al node1..	.	al node2..	al node2..

Total, de peticiones en dos nodos:

- 12414 peticiones/segundo
- 92812 peticiones/10 segundos



El esquema muestra cómo funciona el sistema Redis con dos nodos trabajando en paralelo. Los clientes envían solicitudes a cualquiera de los dos nodos para obtener insultos o filtrar frases, y los nodos responden procesando estas peticiones. El broadcaster se encarga de mantener actualizadas las listas de insultos en ambos nodos. Además, el monitor recopila las estadísticas de cada nodo y las muestra en tiempo real, permitiendo visualizar el rendimiento y la carga de trabajo de todo el sistema.

Hemos realizado algunos tests:

```

✓ redis_node2
> ✓ server1broadcaster_test.py
> ✓ server1client_test.py
> ✓ server1filter_test.py
> ✓ server2broadcaster_test.py
> ✓ server2client_test.py
> ✓ server2filter_test.py
  
```

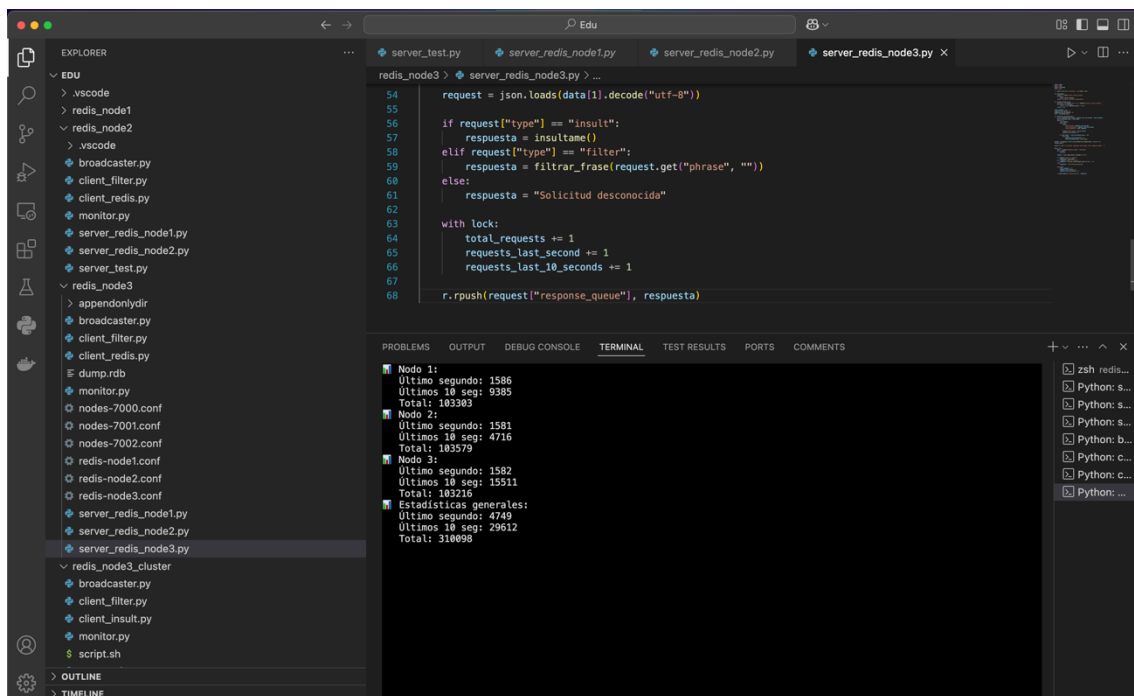
Tres nodos

Añadiendo un **tercer nodo Redis** hemos llevado la arquitectura al siguiente nivel. Ahora el programa cuenta con tres servidores ejecutándose en paralelo, cada uno escuchando sus propias peticiones y respondiendo de forma independiente.

Los **clientes** siguen funcionando igual que antes, pero ahora eligen aleatoriamente entre **tres posibles nodos**. Esto ayuda a repartir mejor la carga y hace que el sistema sea más robusto, sobre todo cuando se lanzan muchos clientes a la vez.

El **broadcaster** también se ha modificado para asegurarse de que todos los insultos llegan a los tres nodos, así todos trabajan con la misma información.

Además, el **monitor** se ha actualizado para mostrar estadísticas de cada nodo por separado y un resumen general con la suma total de peticiones. Esto nos permite ver claramente cómo se reparte el trabajo entre los nodos y comparar su rendimiento en tiempo real.

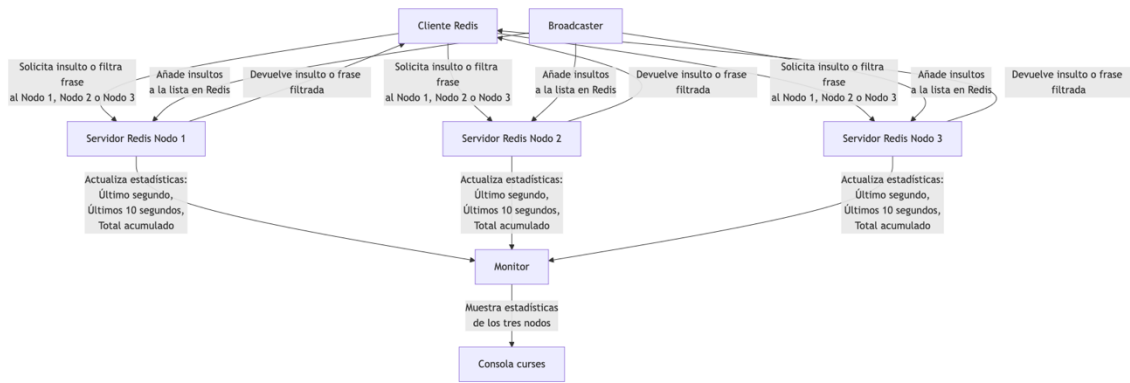


```
54 request = json.loads(data[1].decode("utf-8"))
55
56 if request["type"] == "insult":
57     respuesta = insultame()
58 elif request["type"] == "filter":
59     respuesta = filtrar_frase(request.get("phrase", ""))
60 else:
61     respuesta = "Solicitud desconocida"
62
63 with lock:
64     total_requests += 1
65     requests_last_second += 1
66     requests_last_10_seconds += 1
67
68 r.rpush(request["response_queue"], respuesta)
```

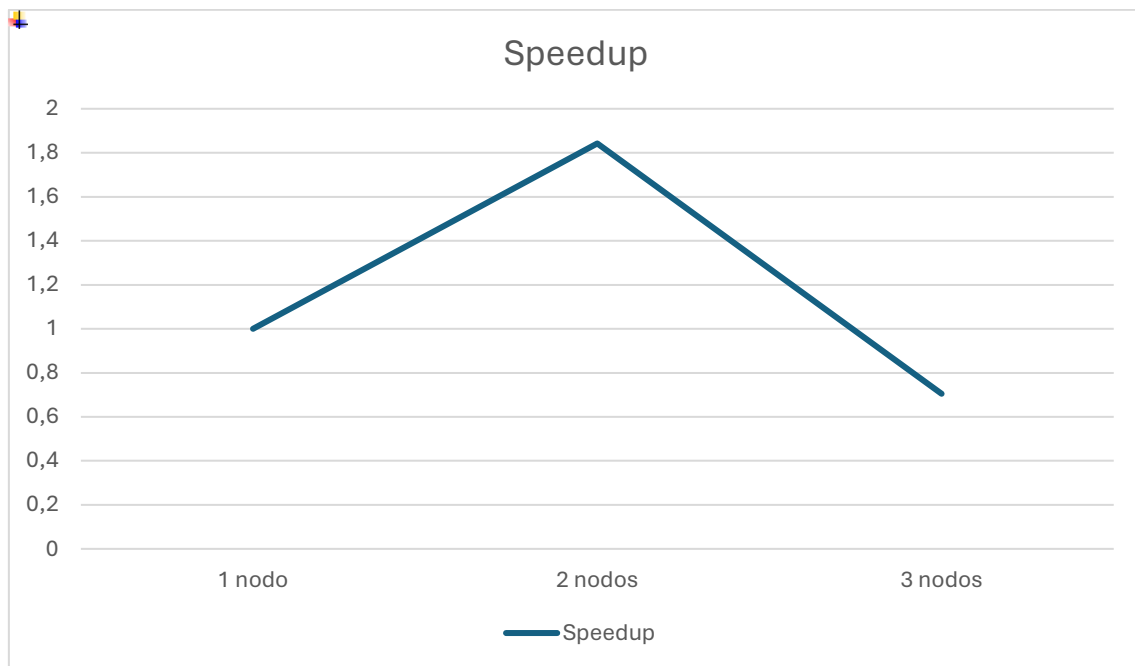
```
Nodo 1:
Último segundo: 1586
Últimos 10 seg: 9385
Total: 183383
Nodo 2:
Último segundo: 1581
Últimos 10 seg: 4716
Total: 183579
Nodo 3:
Último segundo: 1582
Últimos 10 seg: 15511
Total: 183216
Estadísticas generales:
Último segundo: 4749
Últimos 10 seg: 29612
Total: 318098
```

Total, de peticiones en tres nodos:

- 4749 peticiones/segundo
- 29612 peticiones/10 segundos



El esquema muestra cómo funciona el sistema Redis con tres nodos trabajando en paralelo. Los clientes envían solicitudes a cualquiera de los tres nodos para obtener insultos o filtrar frases, y los nodos responden procesando estas peticiones. El broadcaster se encarga de mantener actualizadas las listas de insultos en los tres nodos. Además, el monitor recopila las estadísticas de cada nodo y las muestra en tiempo real, permitiendo visualizar el rendimiento y la carga de trabajo de todo el sistema de manera clara y eficiente.



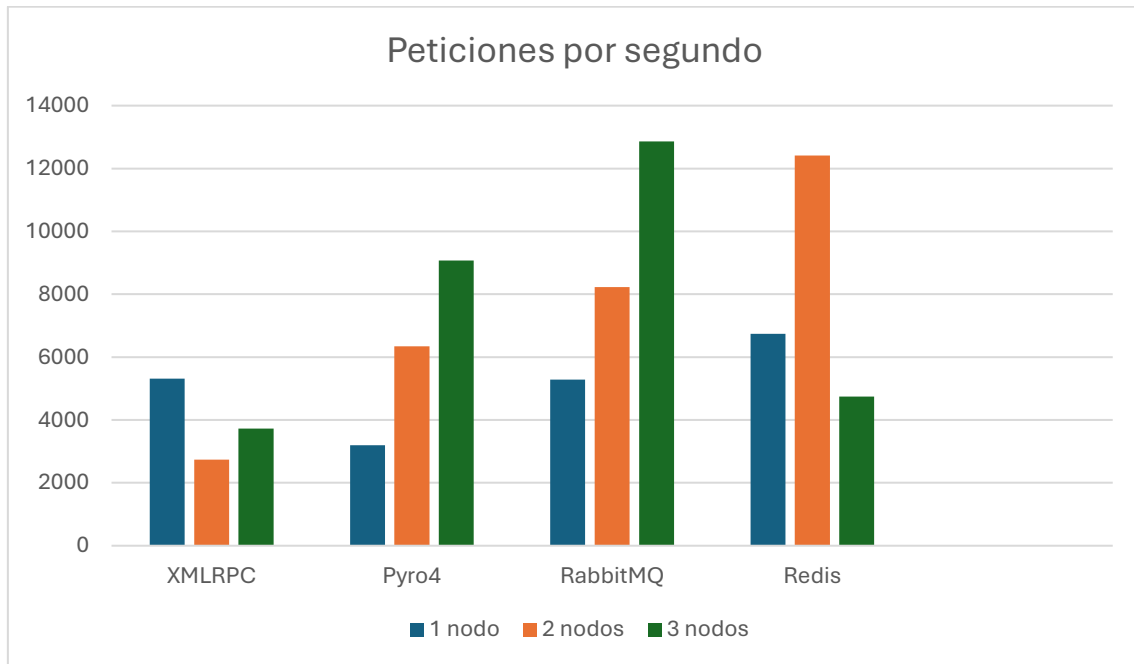
También hemos realizado algunos tests:

✓ 8/8	21.8s ↗	1
✓ redis_node3		2
> ✓ server1broadcaster_test.py		3
> ✓ server1client_test.py		4
> ✓ server1filter_test.py		5
> ✓ server2broadcaster_test.py		6
> ✓ server2filter_test.py	✓	7
> ✓ server3broadcaster_test.py		8
> ✓ server3client_test.py		9
> ✓ server3filter_test.py		10
		11
		12
		13
		14

Análisis de los 4 middleware

Análisis rendimiento (peticiones / segundo)

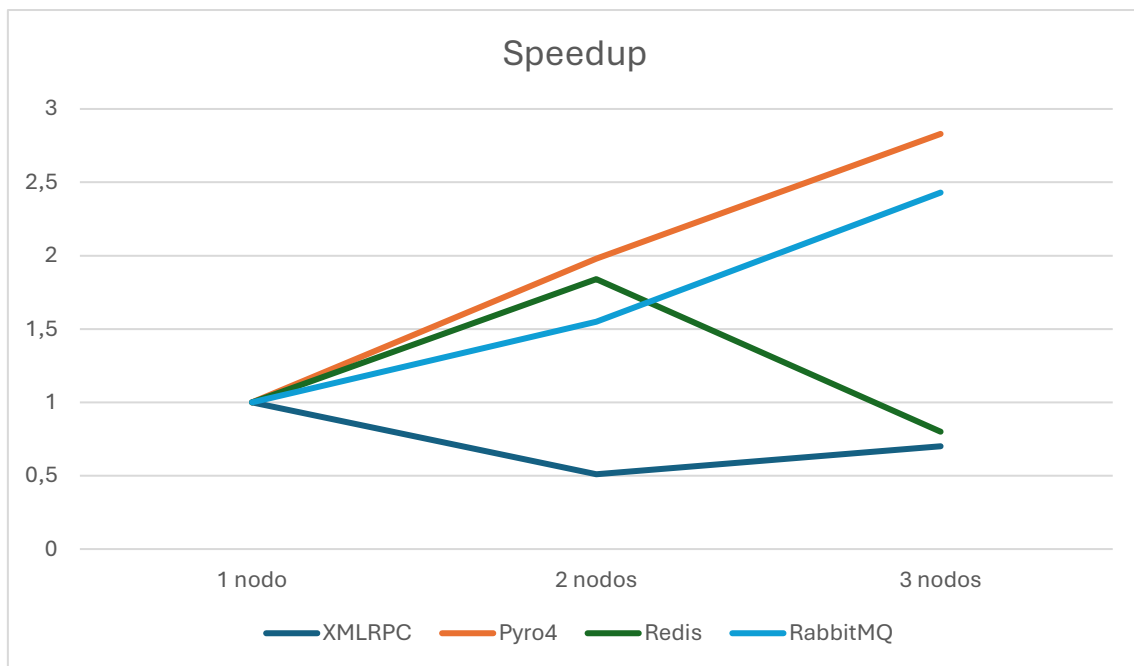
Finalmente, con todos los datos recogidos obtenemos las siguientes estadísticas combinadas.



Observamos 2 situaciones diferentes:

1. El XMLRPC no es capaz para hacer escalado horizontal al igual que le ocurre a Redis a partir de 2 nodos.
2. Vemos que Pyro4 y RabbitMQ sobresalen en el escalado horizontal, siendo Rabbit el completo ganador. Esto nos permite observar a la hora de decidir en distintas situaciones en el mundo laboral cual elegir. Siendo RabbitMQ si necesitamos la máxima estabilidad, y Redis si necesitamos notificaciones rápidas pub/sub.

Análisis y conclusión de los speedup obtenidos



Observamos un crecimiento lineal en Pyro4 y RabbitMQ mientras que aquellos middleware que no escalan bien horizontalmente decae su rendimiento.

Escalado horizontal dinámico

En la práctica se nos propuso hacer un escalado horizontal dinámico. En este caso, lo hemos decidido hacer con Pyro4 ya que es simple y a la vez potente soportando una buena tasa de peticiones por segundo.

Sin embargo, **en las pruebas se observará una clara limitación de mensajes**, esto no es culpa de Pyro4, lo hemos **hecho a propósito ya que ahora el motivo es observar el funcionamiento del escalado horizontal más que la potencia bruta**.

Mientras se mantiene el mismo tipo de sistema distribuido que hemos creado en la práctica con Pyro ahora tenemos:

1. Un escalador que se encarga de analizar los servidores e ir escalando o des escalando horizontalmente. Usando la formula mencionada en la práctica.
 - a. $N = \left\lceil \frac{\lambda \times T}{C} \right\rceil$ lamda = mensajes / segundo, T = tiempo procesamiento por mensaje, C = capacidad de un solo nodo.
2. Un gestor encargado de poder crear o eliminar procesos clones del servidor real.
3. Un archivo TXT donde se guardan los servidores disponibles que luego es consultado tanto por el programa de estadísticas como el cliente.

También hemos tenido que enfrentarnos a diferentes retos, como desestabilizaciones en el escalado / desescalado. Esto ocurre, porque si el gestor detecta que se necesitan añadir 4 servidores y se añaden, luego al estabilizar la carga resulta que no hacían falta tantos servidores por que el umbral de peticiones / segundo entre todos es inferior al umbral necesario para mantenerlos. Esto provoca que el sistema sea inestable, e incluso los clientes tenían tiempos de espera antes de poder seguir mandando peticiones.

Para solucionarlo, finalmente se optó por añadir 2 limitadores para permitir que no pueda escalar tan rápidamente y viceversa.

```
if required_servers > current_count:
    required = required_servers - current_count
    if ((required) > MAX_SCALING_SERVERS):
        required = MAX_SCALING_SERVERS

    for _ in range(required):
        start_server(next_id)
        next_id += 1

elif required_servers < current_count:
    required = current_count - required_servers
    if ((required) > MAX_DOWNGRADING_SERVERS):
        required = MAX_DOWNGRADING_SERVERS
```

Estas variables, actúan como controladores, la lógica de control del numero máximo de servidores permitidos está en el cálculo de la formula.

Aun que el sistema actual utiliza eventos directos con Pyro4 para actualizar los clientes cuando cambia la lista de servidores, este enfoque no es escalable a gran escala (por ejemplo, millones de clientes) por las siguientes razones:

- Pyro4 no está diseñado para manejar millones de conexiones activas.
- El hecho de que los clientes tengan que leer un archivo constantemente puede generar cuellos de botella si la escala crece mucho.
- Registrar a los clientes directamente desde el escalador o mantener conexiones vivas puede hacer que el sistema se vuelva frágil o ineficiente.

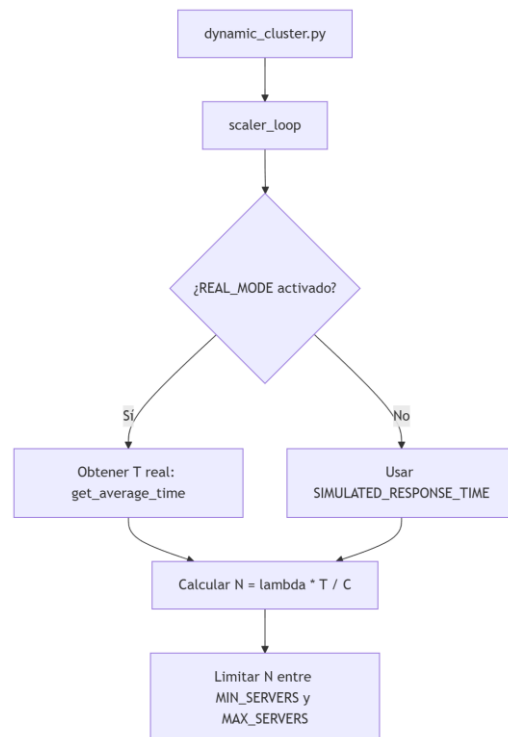
Si quisiéramos un escalado realista podríamos tener:

1. Un modelo de PULL (Service Discovery):
 - a. Los clientes se encargan de consultar periódicamente un servicio central que mantiene la lista de servidores activos.

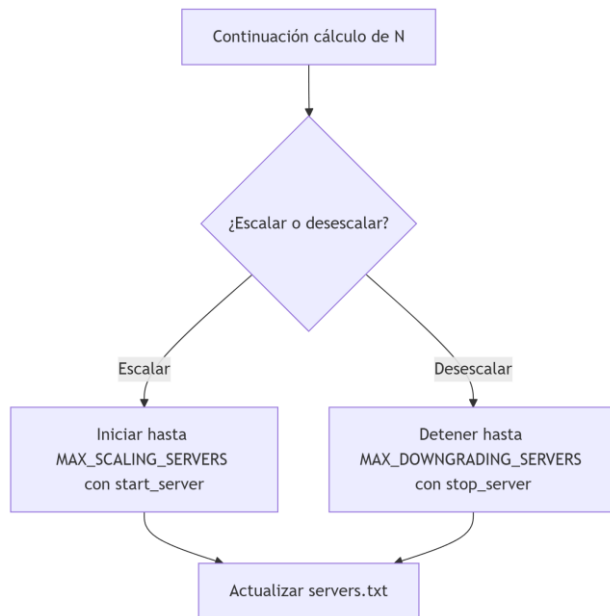
- b. Este servicio puede estar implementado como un microservicio REST, Pyro4 o incluso REDIS si sabemos que se harán muchas consultas.
- 2. Pub/Sub con Redis o ZeroMQ
 - a. El escalador publica mensajes de actualización en un canal.
 - b. Los clientes que se han suscrito recibirán los cambios de forma eficiente y distribuida.
- 3. Uso de balanceadores de carga intermedios.
 - a. Los clientes no acceden a los servidores directamente.
 - b. Se comunican con un balanceador central que redirige las peticiones al servidor menos cargado.
 - c. Esto podríamos implementarlo con un router Pyro4, un reverse proxy o quizás herramientas como HAProxy/Nginx.

Estas alternativas permitirían tener un sistema mucho más escalable, eficiente y preparado para producción real.

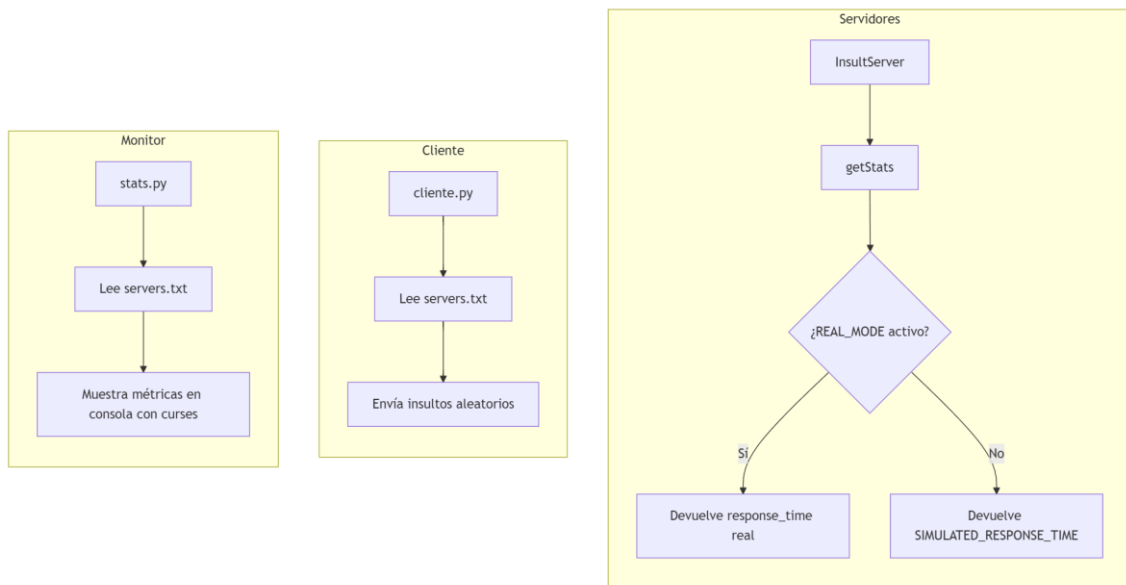
A continuación, presentamos un diagrama tanto del escalador como del gestor:



Profundización cuando se calcula si se escala o desescala y su control.



Ecosistema:



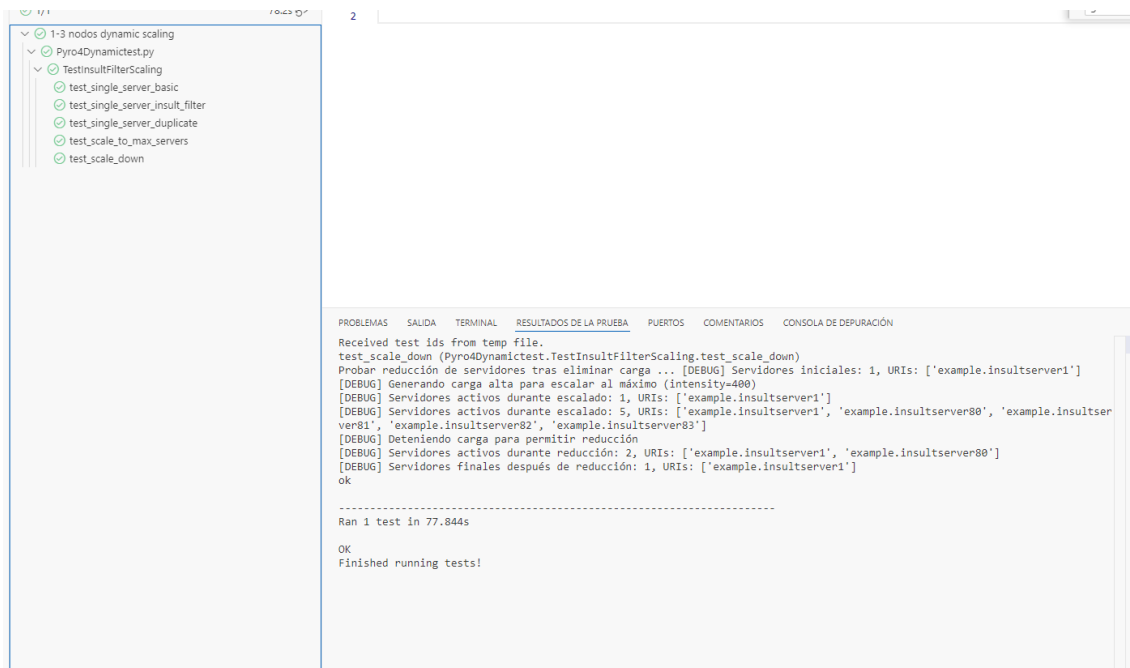
Los cambios realizados han sido en las estadísticas donde ahora también se devuelve el tiempo de respuesta real o el simulado.

También adjuntamos el siguiente video en YouTube [[Escalado Horizontal Pyro4](#)] donde hemos grabado su funcionamiento (los cambios en las estadísticas se hicieron más tarde del video). Estas pruebas han sido grabadas con clientes mandando 10 peticiones / segundo y con la siguiente configuración en el servidor:

```
# ===== CONFIGURACIÓN DE ESCALADO =====  
MIN_SERVERS = 1  
MAX_SERVERS = 5  
SERVER_CAPACITY = 5           # mensajes por segundo por servidor (ajusta también)  
SCALING_INTERVAL = 3 # segundos entre chequeos  
SIMULATED_RESPONSE_TIME = 0.5  
REAL_MODE = False  
MAX_SCALING_SERVERS = 2  
MAX_DOWNGRADING_SERVERS = 1
```

Para que el efecto del escalado fuera más claro, hemos ajustado los umbrales de escalado de una forma un poco más agresiva.

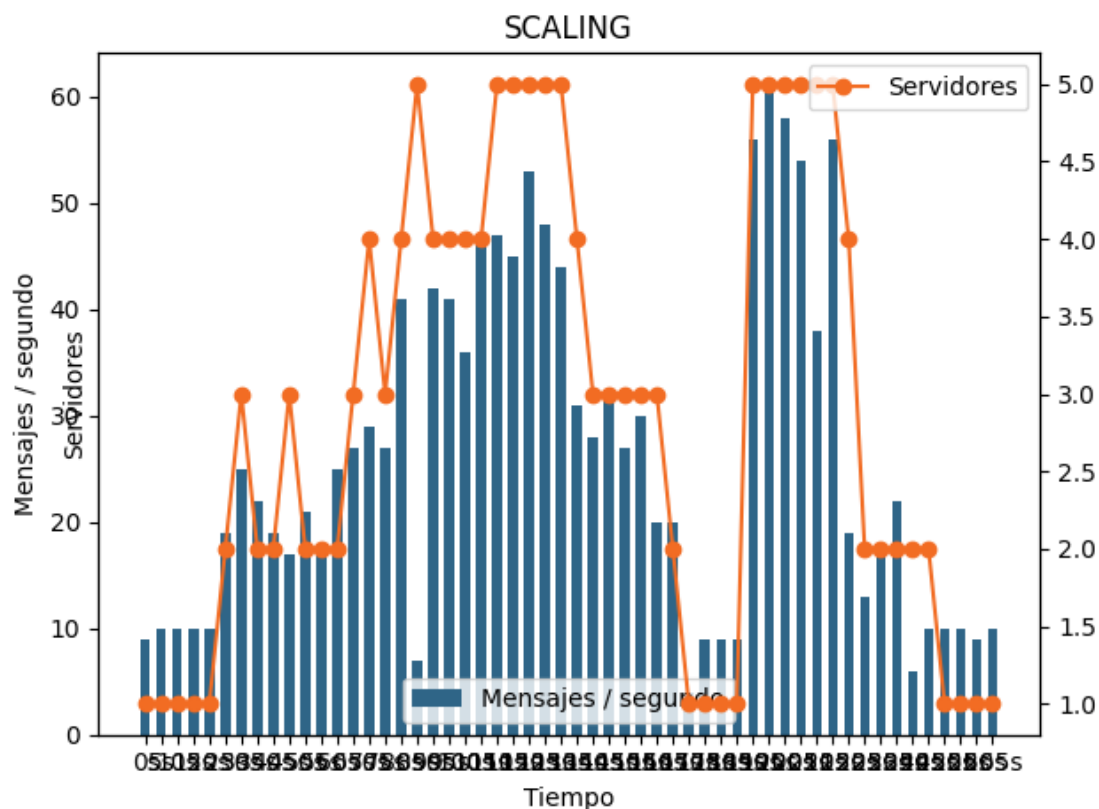
Por parte de las pruebas, se han hecho las siguientes pruebas:



- Pruebas básicas de funcionamiento de 1 nodo (al final los 5 nodos son clones de ese primero).
- Pruebas de escalado horizontal dinámico, es decir, que al estar en gran carga se generen 5 nodos.
- Pruebas de des escalado horizontal dinámico, donde tras estar en una gran carga, comprobamos que vuelva a 1 nodo correctamente.

Grafica.

Además, en las estadísticas hemos añadido una gráfica auto-generada con matplotlib y tk. Esto permite poder observar y documentar el rendimiento del servidor en diferentes momentos de carga. Adjuntamos una de las graficas de 120 segundos donde ponemos al servidor en diferentes situaciones. Recomendamos también probarlo en local, ya que es muy visual y permite ver claramente su funcionamiento.



Observamos como realiza el escalado, tanto en entrada lenta de clientes hasta llegar al máximo de peticiones, como un pico de clientes.

Conclusión: El escalado horizontal dinámico es una herramienta fundamental cuando queremos que nuestro sistema pueda soportar momentos de alta carga sin saturarse. También nos ayuda a optimizar recursos, ya que solo usamos lo necesario en cada momento. Aunque el enfoque de esta práctica está pensado como una demostración, los principios que aplicamos son los mismos que se usan en sistemas reales: monitorizar, escalar y adaptarse. Y eso, al final, es lo que hace a un sistema inteligente.